

Department of Computer Science
Faculty of Applied Sciences
University of Sri Jayewardenepura



Group Project

Name of the program : Flood Relief Management System
Course Module : Fullstack and Fundamentals
Due Date of Submission : 18/ 03 / 2026
Name of the lecturer : Ms. WG Lavanka Harshani
Module No : CSC 1051
Group Number : Group 21

Group Members

- 1) R.G.Y.Sewmith : AS20240555
- 2) K.K.S.Perera : AS20240414
- 3) N.W.W.G.N.M.Nanayakkara : AS20240617
- 4) P.D.A.Weerasinghe : AS20240634

Table of Content

1.Introduction	1
1.1 Brief Description of the Project	1
1.2 Objectives of the System	1
2.System Overview	2
2.1 Overall System Description	2
2.2 Target Users	3
2.3 Technologies Used and How I Integrated Them	3
2.4. Database design (detailed structure of database)	4
2.5. Backend logic	9
2.6. Frontend functionality	15
3.Results	17
3.1.Introduction	17
3.2.System Outputs.....	17
3.3.Screenshots of the system components	18
3.4. Summary of the Project	43

1.Introduction

1.1 Brief Description of the Project

The Flood Relief Management and Analytical Reporting System were created to solve a major problem during natural disasters (much information and not enough clarity). When floods occur, rescue teams are often buried under thousands of requests, making it hard to see who needs help the most. This system acts as a smart filter that turns messy data into clear, organized reports.

A key feature of this project is its "Drill-Down" Dashboard. Instead of forcing an administrator to scroll through endless lists, the dashboard shows a simple summary first. If they want to know more about a specific area or a specific type of aid, they can "drill down" into the details with a single click—without the page ever having to reload. By organizing needs into categories like Food, Water, and Medicine, and ranking them from "High" to "Low" severity, the system makes sure that emergency resources go to the most critical cases first.

The system is built on a reliable and modern foundation:

- **PHP & MySQL:** The "engine" and "memory" that handle all the calculations and store the data safely.
- **HTML, CSS, & Bootstrap:** The "face" of the system, designed to look professional and work perfectly on both computers and mobile phones.
- **JavaScript:** The "brain" behind the interface that makes the tables interactive and fast.

1.2 Objectives of the System

We built this system with four main goals in mind to make disaster response faster and more effective:

- 1.2.1. Creating a Single Source of Truth:** Instead of having data scattered across different spreadsheets or paper notes, this system brings everything into one central database. This prevents confusion and ensures everyone is looking at the same, accurate information.

- 1.2.2. Prioritizing Those in Danger:** Not every request is equally urgent. A primary goal was to build a system that highlights "High Severity" cases. This helps officials send life-saving resources, like boats or medical teams, to the most dangerous zones immediately.
- 1.2.3. Seeing the Big Picture** (and the Small Details): We wanted to give administrators two ways to look at the crisis. They can look at it by Location (which districts are struggling?) or by Resource (do we have enough water for everyone?). This "dual-view" helps manage the supply chain better.
- 1.2.4. Making it Accessible Anywhere:** Disasters don't happen behind an office desk. A major objective was making the system Responsive. By using Bootstrap, we ensured that a field officer standing in the rain can check a report on their tablet or phone just as easily as someone in an office using a desktop.

2. System Overview

2.1 Overall System Description

This platform is a digital command center for managing flood relief efforts. We built it to act as a bridge between the chaotic data coming from the field and the decision-makers in an operations center. The main goal was to solve the problem of "information overload".

Instead of showing a giant, confusing list of every single request, the system organizes everything into clean, high-level summaries.

The standout feature is the "Drill-Down" capability. It allows an administrator to see the big picture (like total requests across a district) and then, with one click, dive into the specifics (like how many of those requests are for urgent medical supplies). It's designed to turn messy data into a clear plan of action.

2.2 Target Users

We kept two specific groups of people in mind while building this:

1. **Disaster Management Admins:** These are the people in charge during a disaster. They need a clear overall view of the situation. They use reports to find out which districts are most affected and then send help, such as food trucks and rescue boats, to the areas that need it most.
2. **Users:** These are the teams on the ground. Because the system is responsive, they can check the latest requirements or update statuses right from their tablets or phones while they are out in the field.

2.3 Technologies Used and How I Integrated Them

To make the system fast and reliable, We chose a classic but powerful "Client-Server" stack and tied them together like this:

1. **The Backend (PHP & MySQL):** We used PHP to handle all the "heavy lifting" logic (like calculating request totals and filtering data) when a user clicks a button. MySQL acts as the system's memory, storing every request, location, and severity level in organized tables so they can be retrieved instantly.
2. **The Interface (HTML & CSS):** These were used to build the skeleton and the visual style of the dashboard, making sure it looks professional and is easy to read under pressure.
3. **The Interactivity (JavaScript):** This is the "brain" of the frontend. I integrated JS to handle the view-switching logic. This is what allows the "View More" and "Close" buttons to work instantly without the page having to reload, making the app feel snappy and modern.
4. **The Framework (Bootstrap):** I integrated Bootstrap to make the system "device-agnostic." It automatically adjusts the layout so that the reports look just as good on a small smartphone as they do on a large office monitor.

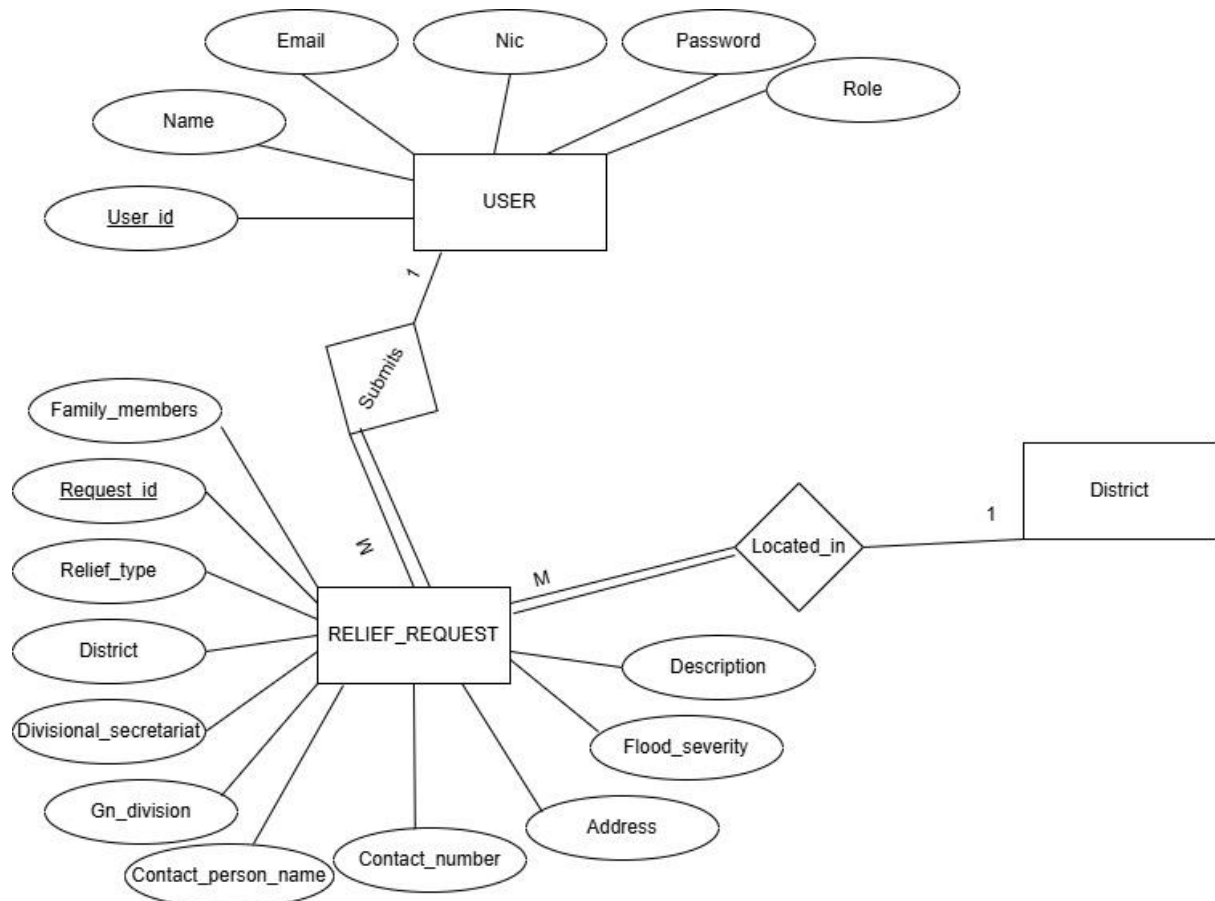
2.4. Database design (detailed structure of database)

2.4.1. Overview of the Database

The database of the Flood Relief Management System is designed to efficiently store, manage, and retrieve information related to flood disaster response activities. It acts as the central repository for all data required to coordinate relief efforts, manage affected individuals, allocate resources and monitor relief operations. The database ensures that information is organized systematically, allowing administrators to access accurate and up-to-date data when making decisions during emergency situations. The system stores different types of information such as details of flood victims and distribution of relief supplies. Each category of information is stored in separate but related tables to maintain data consistency and reduce redundancy. These tables are connected using primary keys and foreign keys, allowing the system to establish relationships between different entities within the database.

Through this structured design, the database enables efficient tracking of affected people, monitoring of available resources, and proper allocation of aid to those in need. By maintaining accurate records and supporting quick data retrieval, the database plays a critical role in improving the overall efficiency and coordination of flood relief management activities.

2.4.2. Entity–Relationship (ER) Diagram



2.4.3. Table Structure

Table: User

The User table stores the details of individuals who register in the system. Each user is assigned a unique user ID which acts as the primary key. This table contains personal information such as name, email, NIC number and password, allowing the system to authenticate and manage users efficiently.

Attributes	Data Type	Key	Description
user_id	INT	Primary Key	Unique ID for each user
name	VARCHAR(100)		User's full name
email	VARCHAR(100)		User's email address
NIC	VARCHAR(12)		User's NIC
password	VARCHAR(10)		User password

Table: Request

The Request table stores all relief requests submitted by affected individuals. Each request is uniquely identified by a request ID. The table also contains details such as the type of relief required, location information, severity level, and the current status of the request(whether it is accepted or rejected).

Attributes	Data Type	Key	Description
request_id	INT	Primary Key	Unique ID for each request
user_id	INT	Foreign Key	Connect with User table
relief_type	ENUM		Food, Shelter, Medicine, Water
district	VARCHAR(100)		Affected district
dev_secretariat	VARCHAR(100)		Divisional secretariat

gn_devision	VARCHAR(100)		Grama niladhari division
contact_name	VARCHAR(100)		Name of the person who make requests
contact_number	VARCHAR(100)		Phone number of the person who make requests
address	TEXT		Address of the affected place
family_members	INT		Number of members in the family
flood_severity	ENUM		High, Medium, Low
description	TEXT		Description of details
status	ENUM		Pending, Accepted, Rejected
created_at	TIMESTAMP		Time the request made

2.4.4. Relationships Between Tables

In the Flood Relief Management System, the database tables are connected through relationships that help organize and manage data efficiently. These relationships ensure that the information stored in different tables is properly linked and can be retrieved easily when required.

The main relationship in this system exists between the User table and the Request table. A single user can submit multiple relief requests based on their needs during a flood disaster. However, each request is associated with only one specific user who submitted that request. Therefore, this relationship is defined as a one-to-many relationship.

This relationship is implemented using the user_id attribute. In the User table, the user_id acts as the Primary Key, uniquely identifying each user. In the Request table, the same user_id is used as a Foreign Key to connect each request to the corresponding user.

By establishing this relationship, the system can efficiently track which user submitted each request and manage multiple requests from different users. This also helps maintain data integrity, prevent inconsistencies, and improve the overall organization of the database.

2.4.5. Data constraints

Data constraints are rules applied to database tables to ensure the accuracy, reliability, and consistency of stored data. These constraints help maintain the integrity of the database by preventing invalid or duplicate data entries.

In the Flood Relief Management System, several constraints are used to control how data is stored. The Primary Key constraint ensures that each record in a table has a unique identifier so that individual records can be easily distinguished. The Foreign Key constraint is used to establish relationships between tables, linking related data across the database. The NOT NULL constraint ensures that essential fields cannot be left empty, which helps prevent incomplete records. Additionally, the UNIQUE constraint prevents duplicate values from being stored in specific columns, such as email addresses, ensuring that each entry remains distinct.

These constraints play an important role in maintaining the structure and reliability of the database, allowing the system to manage flood relief information efficiently.

- **Primary Key** – ensures unique records
- **Foreign Key** – maintains relationships
- **NOT NULL** – prevents missing important data
- **UNIQUE** – prevents duplicate entries

2.5. Backend logic

2.5.1.Overview of Backend Logic

The backend logic of the Flood Relief Management System is responsible for processing data and managing the communication between the user interface and the database. When a user submits a relief request through the frontend interface, the backend validates the input data to ensure that all required fields are correctly filled. The validated data is then processed using PHP scripts and stored in the MySQL database.

The backend also performs database operations such as creating, retrieving, updating, and deleting records (CRUD operations). Additionally, it retrieves stored data to generate reports and summaries that help administrators monitor relief requests and make informed decisions during flood management activities.

2.5.2.Core Functionalities

1.User Management:

- Add, update and delete records

2.Request Management:

- View, update and delete requests

3.Reporting and Monitoring:

- Generate reports for administrators
- Monitor ongoing operations

2.5.3.Data Flow and Logic

Input – Processing – Output

- Validates data and assigned IDs
- Updates the database
- Confirmation is sent back to the user interface
- Avoid duplicate entries

Example: This query retrieves all records from the Request table where the status is set to 'Pending' allowing the admin dashboard to display new requests for review

```
$user_id      = $data["user_id"];
$relief_type   = $data["relief_type"];
$district     = $data["district"];
$div_secretariat = $data["div_secretariat"];
$gn_division  = $data["gn_division"];
$contact_name = $data["contact_name"];
$contact_number = $data["contact_number"];
$address      = $data["address"];
$family_members = $data["family_members"];
$flood_severity = $data["flood_severity"];
$status       = $data["status"];
$description   = $data["description"];

$sql = "INSERT INTO request (user_id,relief_type,district,div_secretariat,gn_division,contact_name,contact_number,
address,family_members,flood_severity,status,description ) VALUES
('$user_id','$relief_type','$district','$div_secretariat','$gn_division','$contact_name','$contact_number',
'$address','$family_members','$flood_severity','$status','$description' )";
$stmt = mysqli_query($conn,$sql);

if ($stmt) {
    echo json_encode(value: ["status" => "Success","message" => "Request Added"]);
} else {
    echo json_encode(value: ["status" => "error",
    "message" => "Database error",
    "debug" => mysqli_error($conn)]);
}
```

2.5.4. Security and Access Control

- Admins can view/update/delete data
- Public users may only view certain information

2.5.5.Integration with database

- Ensures data consistency and prevents errors
- **CRUD operations:** Create, Read, Update, Delete data in tables

1.CREATE

This query represents the "Create" part of the CRUD operations, specifically when an affected user submits the relief request form. When the user clicks "Submit," the PHP backend triggers this SQL command to write a new row into the requests table. It captures essential data like the user's ID, the type of aid needed, and the geographic location. The logic ensures that all "NOT NULL" constraints are met before the data is committed to the database. By using this query, the system officially records a person's need, assigning it a unique request_id.

Example: Registration Query

```
$name = $data['name'];
$email = $data['email'];
$password = $data['password'];
$NIC = $data['NIC'];

if( $name == "" || $password == "" || $email == "" || $NIC == ""){
    echo json_encode(["All required fields must be filled."]);
    exit();
}

$hashed_password = password_hash($password, PASSWORD_DEFAULT);

$sql = "INSERT INTO user(name,email,password,NIC)
VALUES ('$name', '$email', '$hashed_password', '$NIC')";

$result = mysqli_query($conn, $sql);

if ($result){
    echo json_encode(["message" => "success"]);
} else {
    echo json_encode(["message" => "failed.", "error" => mysqli_error($conn)]);
}
```

2.READ

The "Read" operation is the most frequently used logic in the system, as it allows administrators to view and analyze incoming data. This specific query retrieves all records from the requests table where the status is currently set to 'Pending'.

Example:

Displaying Request (User dashboard):

```

$user_id = $data["user_id"];
$sql = "SELECT * FROM request WHERE user_id='$user_id'";
$result = mysqli_query($conn,$sql);
$requets = [];

while ($row = $result->fetch_assoc()) {
    $requets[] = $row;
}

echo json_encode($requets);
exit;

```

Displaying All Requests (Admin Dashboard):

Here all the data is read by one query and then the necessary data are taken.

```

$result = $conn->query("SELECT * FROM user");
$users = [];

while ($row = $result->fetch_assoc()) {
    $users[] = $row;
}

echo json_encode($users);
exit;

```

Retrieving User Name for the User Dashboard:

```

$id = $data['user_id'];
$result = $conn->query("SELECT name FROM user WHERE user_id='$id'");
$name = $result->fetch_assoc();
echo json_encode($name);
exit;

```

3.UPDATE

The "Update" operation is used when an administrator makes a decision on a pending aid application. This query modifies the status column of a specific record, changing it from 'Pending' to either 'Accepted' or 'Rejected'.

Example:

Updating Status:

```
$request_id      = $data["request_id"];
$status          = $data["status"];

$sql = "UPDATE request SET status='$status' WHERE request_id='$request_id'";
$result=mysqli_query($conn,$sql);

if($result){
    echo json_encode(["message" => "Request Updated"]);
    exit;
}else {
    echo json_encode(["message" => "Error"]);
    exit;
}
```

Updating User Details:

```
$request_id      = $data["request_id"];
$relief_type     = $data["relief_type"];
$district        = $data["district"];
$div_secretariat = $data["div_secretariat"];
$gn_division     = $data["gn_division"];
$contact_name    = $data["name"];
$contact_number  = $data["contact_number"];
$address         = $data["address"];
$family_members  = $data["family_members"];
$flood_severity  = $data["flood_severity"];
$description     = $data["description"];

$sql = "UPDATE request SET relief_type='$relief_type',district='$district',div_secretariat='$div_secretariat',gn_division='$gn_division',contact_name='$contact_name',
contact_number='$contact_number',address='$address',family_members='$family_members',flood_severity='$flood_severity',description='$description' WHERE request_id='$request_id'";
$result=mysqli_query($conn,$sql);

if($result){
    echo json_encode(["message" => "Request Updated"]);
    exit;
}else {
    echo json_encode(["message" => "Error"]);
    exit;
}
```

4.DELETE

Deleting Users:

The "Delete" operation is used for administrative maintenance, such as removing duplicate accounts.

```
$id = $data["id"];

$sql = "DELETE FROM user WHERE user_id='$id'";
$result=mysqli_query($conn,$sql);

if($result){
    echo json_encode(["message" => "Request Deleted"]);
    exit;
}else {
    echo json_encode(["message" => "Error"]);
    exit;
}
```

Deleting Requests (By Users):

```
$id = $data["id"];

$sql = "DELETE FROM request WHERE request_id='$id'";
$result=mysqli_query($conn,$sql);

if($result){
    echo json_encode(["message" => "Request Deleted"]);
    exit;
}else {
    echo json_encode(["message" => "Error"]);
    exit;
}
```

2.6. Frontend functionality

2.6.1. Overview of the Frontend

The frontend functionality represents the visual interface of the Flood Relief Management System through which users interact with the application. It provides forms, dashboards, and reporting pages that allow users to submit relief requests, view the status of their requests, and manage related information.

The frontend is developed using HTML and CSS to structure and style the web pages, while JavaScript is used to add interactivity and dynamic behavior. In addition, the Bootstrap framework is used to create a responsive design, ensuring that the system can be easily accessed on different devices such as desktops, tablets, and mobile phones. This user-friendly interface helps users efficiently navigate the system and perform required tasks.

2.6.2. User Interface Components

- Dashboard:

Request page

- My request page:

View requests

Update requests

Delete requests

- Admin page

Details : 1.View – pending requests

2.Accept requests

3.Reject requests

4.View the count of – registered users

5.Food requests

6.Medicine requests

7.Shelter requests

8.High severities

Summary

Area wise analysis

Relief Type analysis

Severity Analysis

Search

District wise searching

Relief wise searching

Severity wise searching

2.6.3. Interaction with backend

Submitting forms – backend validates and updates database

Requesting data – Backend fetches info and sends it to frontend

3.Results

3.1.Introduction

The Results section presents the outputs generated by the Flood Relief Management System, including screenshots of key system components and modules. It demonstrates how the system handles data input, processes information, and provides useful outputs for administrators, volunteers, and other stakeholders. This section also summarizes the overall performance and effectiveness of the system in supporting flood relief operations.

3.2.System Outputs

The system outputs present the processed information generated by the Flood Relief Management System. These outputs include updates on request status, analytical summaries, and statistical reports that help monitor relief activities.

Administrators can view different summaries such as **area-wise analysis**, **relief type analysis**, and **severity analysis**. These outputs provide a clear overview of the disaster situation, enabling decision-makers to identify urgent needs and allocate relief resources effectively during flood management operations.

- Requests – shows whether accepted or rejected
- Summary – Area wise, Relief wise, Severity wise summaries

3.3. Screenshots of the system components

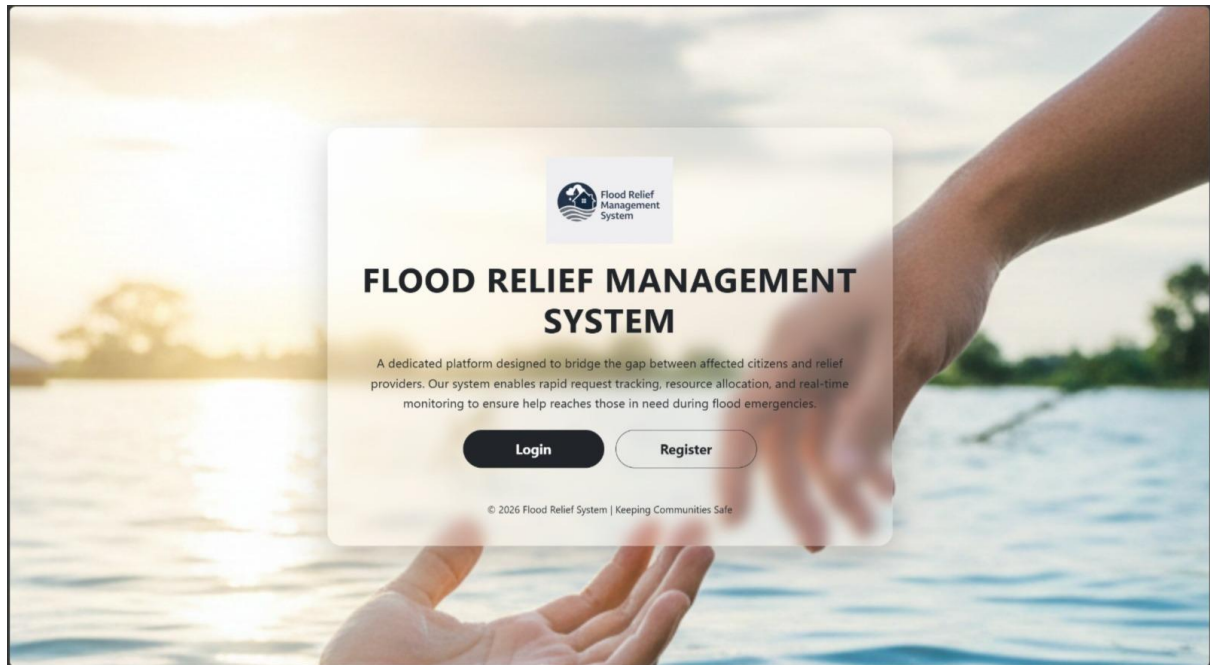


Figure 1: Login and Registration Interface.

This figure displays the home page of the Flood Relief Management System, serving as the primary entry point for all users. The design is intentionally simple and user-friendly to allow individuals in high-stress situations to navigate the system with ease. It provides two distinct options: "Login" for returning users and "Register" for new ones. The interface includes a brief description of the system's mission to connect affected citizens with relief providers. This page acts as a secure gateway, ensuring that all data entered is tied to a verified account. It is built using HTML and CSS for a professional look and Bootstrap to ensure it is responsive on all devices. By requiring authentication, the system maintains a "single source of truth" for disaster data. The page's clean layout helps reduce information overload from the very first interaction. It serves both disaster management admins and general users effectively. This initial screen is critical for the overall security and access control of the platform.

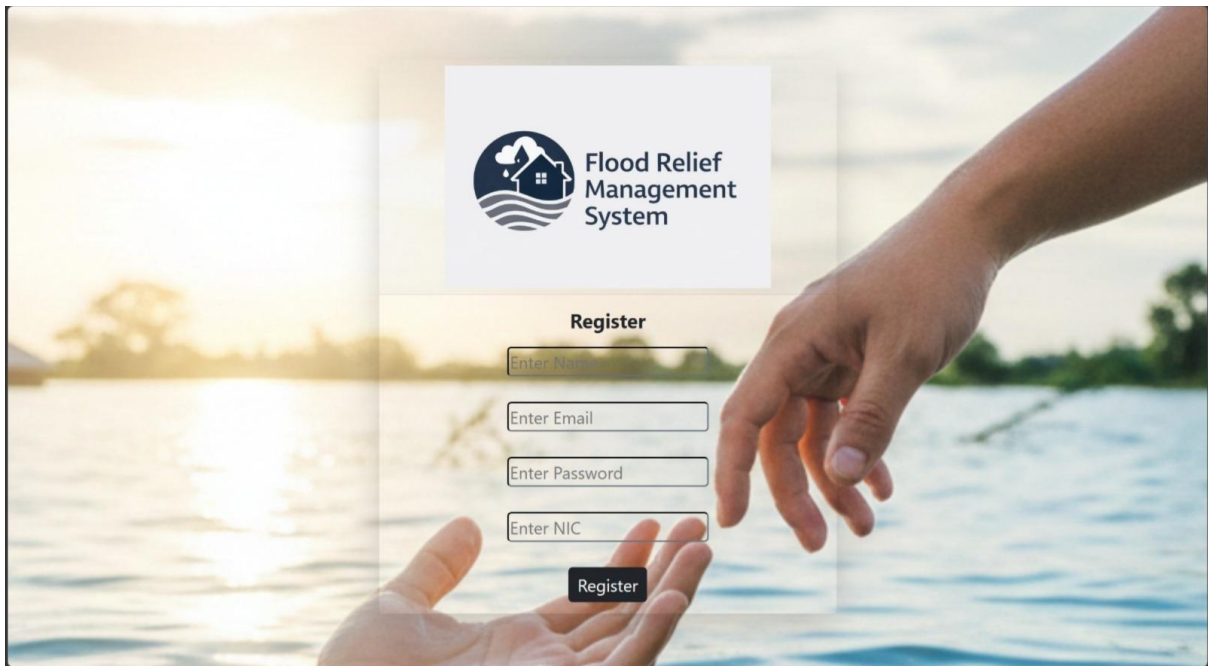


Figure 2: User Register Interface

The User Register Interface is where new individuals provide their details to create a unique account in the system. It collects vital personal information, including the user's full name, email address, and NIC number for identification. Each user is assigned a unique `user_id`, which serves as the primary key in the "User" database table. This table allows the backend to authenticate and manage users efficiently throughout the relief process. The registration form is a key part of the frontend functionality, designed to be accessible on smartphones and desktops alike. Once submitted, the backend PHP scripts validate the data before storing it in the MySQL database. This process ensures that every relief request can be traced back to a specific, verified person. It helps prevent duplicate entries and ensures the reliability of the system's data. The interface is structured to be fast, minimizing the time needed for users to join the platform. This module is essential for building the user-base that the relief system serves.

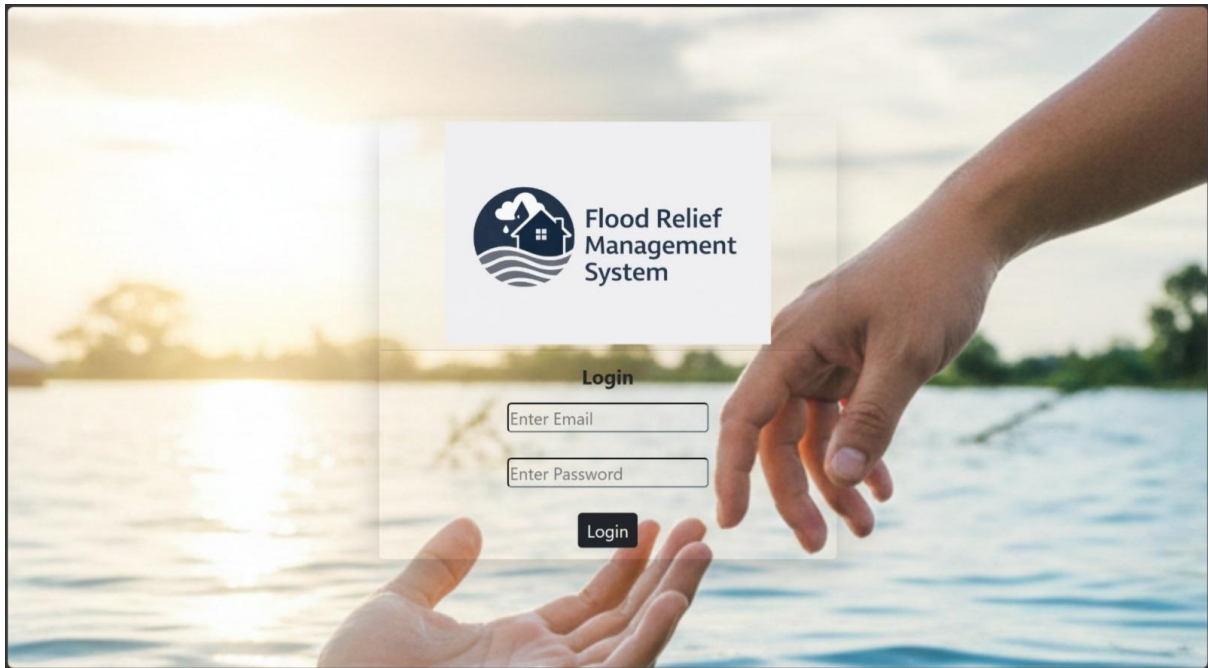


Figure 3: User login

The User Login interface allows registered members to securely access their accounts by entering their credentials. This module is the second half of the authentication system, cross-referencing user input with the "User" table in the database. Successful login is required before a user can submit or manage any relief requests. The backend logic here handles security and access control, distinguishing between general users and administrators. This page is developed with HTML, CSS, and Bootstrap to maintain a consistent, professional visual style. It plays a major role in data privacy by ensuring users can only see their own request history. The interaction is made snappy and interactive through the use of JavaScript. This interface is a critical checkpoint that ensures the integrity of the data being managed. It simplifies the user experience by providing a direct path to the "My Request" and "Update Requests" pages.

The screenshot displays the 'Relief Request Form' within the 'Flood Relief Management System'. The interface is divided into a sidebar on the left and a main content area. The sidebar contains a 'Request' button (active) and a 'View Request' button. The main content area features a form with the following sections:

- Relief Request Form Header:** A dark header bar with the title 'Relief Request Form'.
- Type of Relief Needed:** A dropdown menu with the placeholder text 'Choose one...'.
- District:** A dropdown menu with the placeholder text 'Select District'.
- Divisional Secretariat:** A text input field with the placeholder text 'Enter DS Division'.
- GN Division:** A text input field with the placeholder text 'Enter Grama Niladhari Division'.
- Household Details:** A section containing:
 - Contact Person Name:** A text input field.
 - Contact Number (10 digits):** A text input field with the placeholder text '07XXXXXXX'.
 - Address:** A text input field.
 - No. of Family Members:** A text input field.
- Flood Severity Level:** A dropdown menu with the placeholder text 'Choose one...'.
- Additional Description:** A text area with the placeholder text 'Add any other details...'.
- Logout Button:** A red button with a white arrow icon and the text 'Logout'.

Figure 4:Relief Request Interface

This interface is the core form used by affected individuals to request specific types of assistance. Users can select their needs from categories such as Food, Shelter, Medicine, and Water via an ENUM selection. It captures detailed location information, including the District, Divisional Secretariat, and GN Division. A critical feature of this form is the "Flood Severity" field, which allows users to rank their situation as High, Medium, or Low. The system also asks for the number of family members and a contact name to help coordinate the exact amount of aid. All data submitted through this interface is sent to the "Request" table in the MySQL database. Each submission is automatically stamped with the time it was made using the created_at field. The form is designed with "NOT NULL" constraints to ensure that no essential information is missing. This interface is the primary source of the "Big Picture" data used by administrators to plan relief efforts.

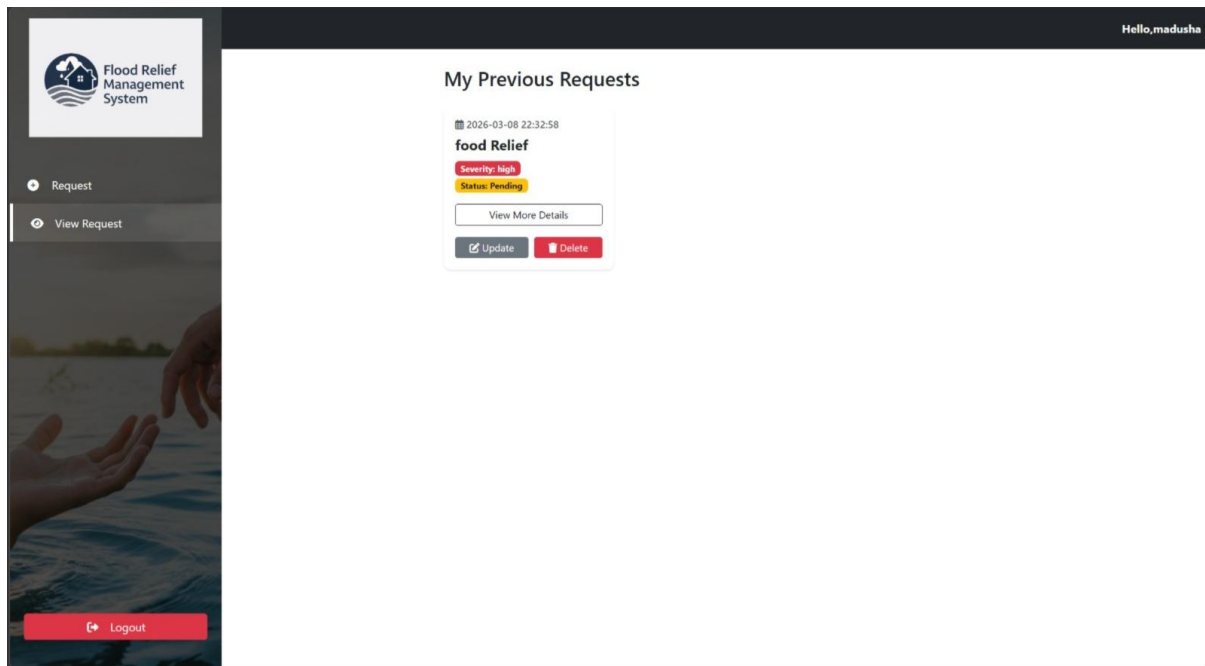


Figure 5:View requests

The View Requests interface provides users with a transparent list of all the aid applications they have submitted. This page allows users to monitor the current status of their requests, seeing whether they are "Pending," "Accepted," or "Rejected". The backend fetches this data by linking the user's user_id to the corresponding records in the Request table. It serves as a personal dashboard where users can track the progress of the assistance they need. The layout is organized into an easy-to-read table format, often built with Bootstrap for responsiveness. Users can see the type of relief requested and the date it was submitted at a glance. This interface helps reduce the psychological stress on victims by providing clear, real-time updates. It is an essential part of the "My Request" page functionality. The page ensures that users feel heard and that the system is actively working on their behalf.

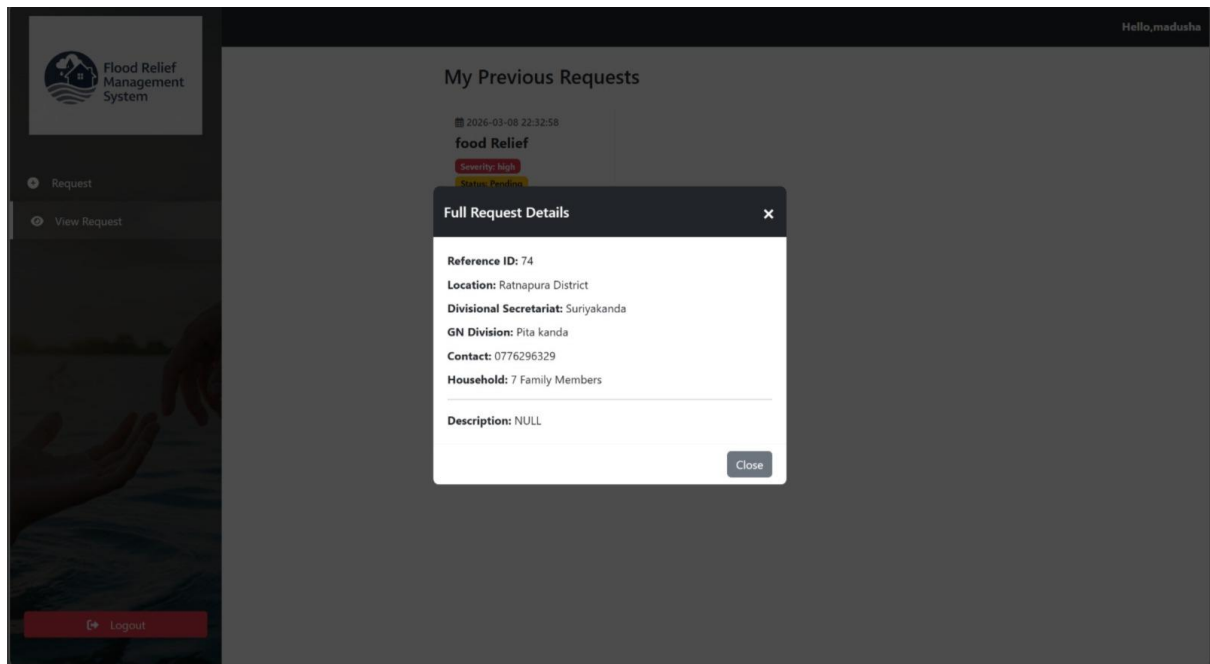


Figure 6:Full request details

This interface offers an expanded, detailed view of a single relief request, providing all submitted information in one place. It displays the complete address, contact numbers, and the specific description provided by the user. This detailed view is vital for ensuring that there is no confusion regarding the location or the family's specific needs. The backend retrieves this information from the database using the unique request_id. Administrators and field officers use this screen to verify the legitimacy and urgency of a request before acting. It highlights the severity level prominently so that life-saving resources can be prioritized. The interface is clean and uncluttered, designed for high readability under pressure. This module is part of the system's objective to move from a general summary to specific, actionable data. It provides the "small details" necessary for the actual delivery of food or medical supplies.

The screenshot shows a web application interface for a Flood Relief Management System. On the left is a sidebar with a logo and navigation links for 'Request' and 'View Request', along with a 'Logout' button at the bottom. The main area displays a modal window titled 'My Previous Requests' with a sub-header 'Update Request'. The form within the modal contains the following fields: 'Relief Type' (dropdown menu set to 'Food'), 'District' (dropdown menu set to 'Ratnapura'), 'Divisional Secretariat' (text input set to 'Suriyakanda'), 'GN Division' (text input set to 'Pita kanda'), 'Contact Person Name' (text input set to 'Saman Kumara'), 'Contact Number' (text input set to '0776296329'), and 'Address' (text input set to '96/5,B, pitakanda, buluthota'). At the bottom right of the form are two buttons: 'Close' and 'Save Changes'.

Figure 7:Update Requests

The Update Requests interface allows users to modify the details of an existing request if their situation changes. For example, if floodwaters rise, a user can change the severity level from "Medium" to "High" to alert administrators. The page pre-populates the form with existing data, allowing for quick edits to fields like address or contact number. This functionality is part of the system's CRUD (Create, Read, Update, Delete) operations handled by the backend. Updating a request instead of creating a new one helps maintain data integrity and prevents duplicate entries in the database. The system ensures that the unique request_id remains the same while the data within it is refreshed. This empowers the user to keep their "Single Source of Truth" up to date as the disaster unfolds. The interface is responsive, so updates can be made from the field via a mobile phone. This feature is a core component of the "My Request" management section for general users

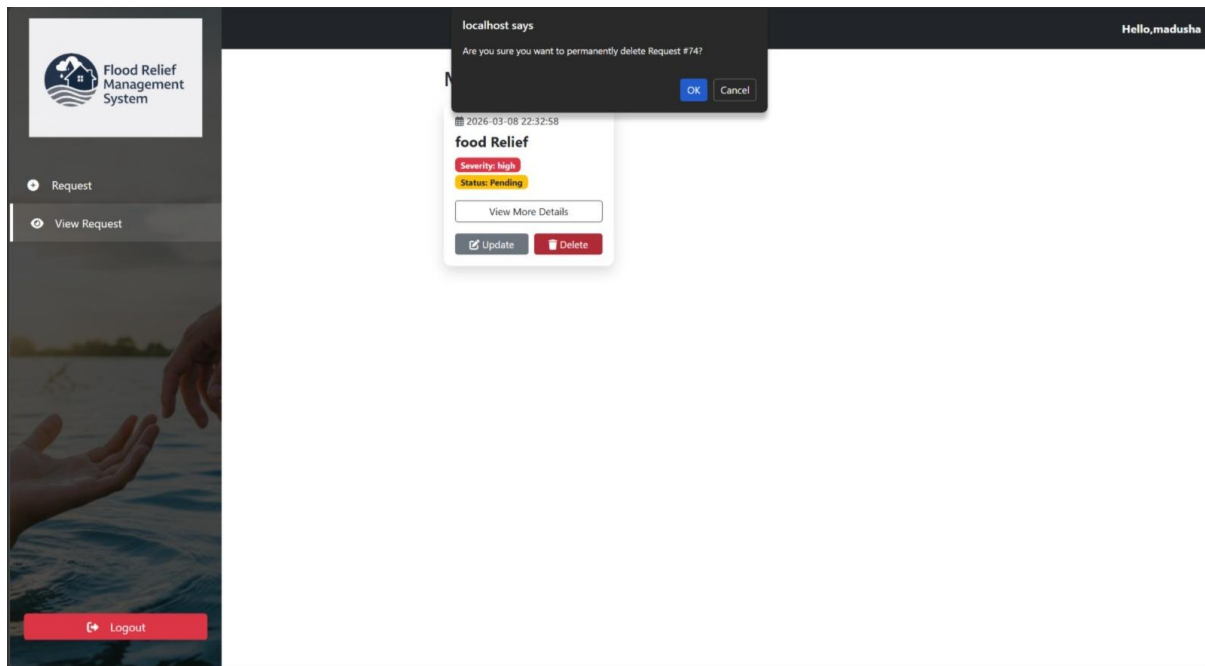


Figure 8: User Request History and Status Tracking

This interface serves as a historical log and a tracking system for all interactions between the user and the relief management. It consolidates all past and present requests into a single view for long-term monitoring. The "Status" column is the most important element, showing if a request is "Pending," "Accepted," or "Rejected". The backend dynamically updates this view whenever an administrator changes the status of a request. This transparency builds trust between the affected citizens and the disaster management teams. The layout is designed to be accessible, following the professional styling used throughout the dashboard. It allows users to confirm that their updates have been correctly processed by the system. This view is critical for the "Target Users" on the ground to see the status of their neighborhood's aid. It embodies the system's objective of making critical data accessible anywhere.

Admin

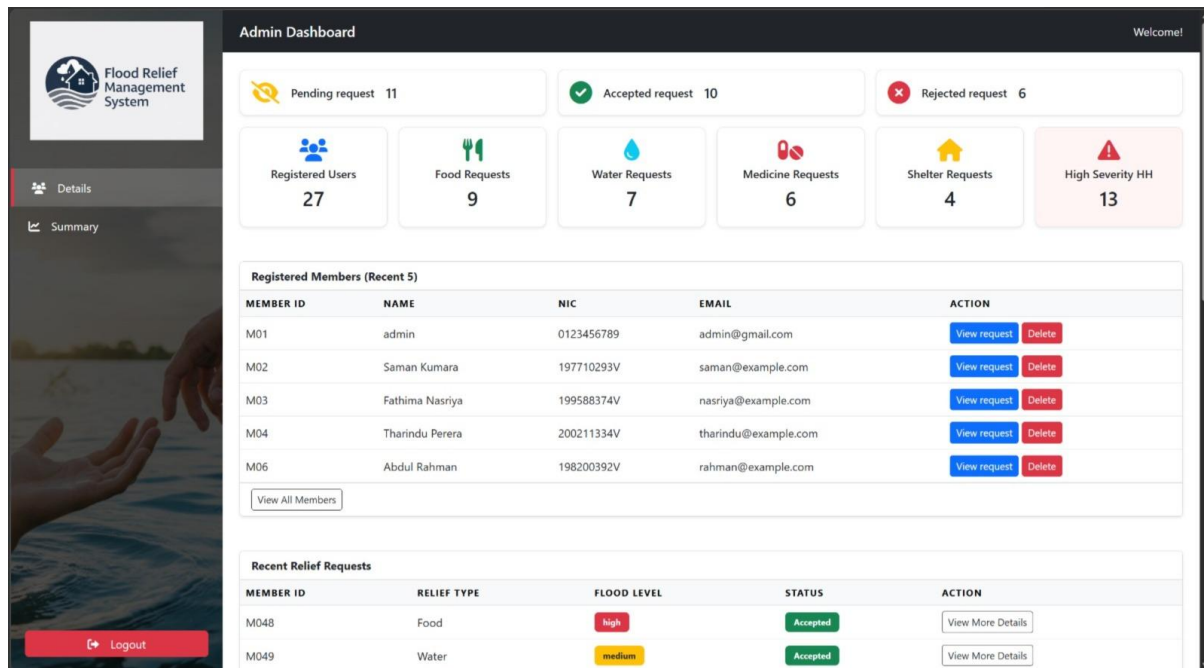


Figure 9: Admin Dashboard Overview

The Admin Dashboard Overview is the central "Command Center" for disaster management administrators. It provides high-level summary cards that display the total number of registered users and the count of relief requests. The dashboard also categorizes requests by type, showing how many individuals need Food, Water, Medicine, or Shelter. A primary focus of this screen is the "High Severity" count, which highlights the most critical cases immediately. This interface is designed to solve the problem of "information overload" by turning thousands of data points into a clear summary. It utilizes "Drill-Down" capabilities, allowing admins to click a summary card to see the detailed list behind the numbers. The backend PHP scripts perform real-time calculations to keep these totals accurate. The dashboard's professional visual style is created with Bootstrap to ensure it works on office monitors and field tablets. This is the primary tool for administrators to "See the Big Picture" of the disaster.

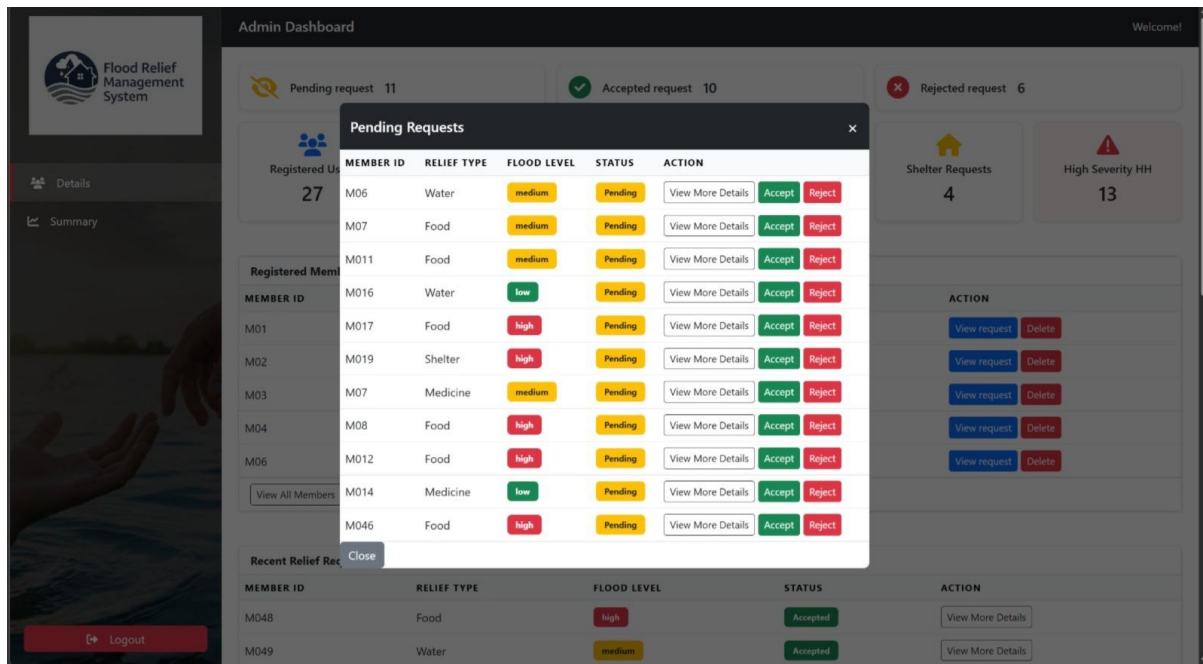


Figure 10: Pending requests

This interface lists all relief requests that have been submitted but are currently awaiting a decision. It serves as a work queue for administrators, ensuring that every request is reviewed in a systematic way. Each row in the table provides the requester's name, their district, and the urgency level. The backend filters the MySQL database to show only records where the status is set to "Pending". Admins can click on a specific request to "drill down" into the full details before making a choice. This page is vital for prioritizing help, as it allows admins to identify high-severity cases within the pending list. It maintains the "Single Source of Truth" by ensuring that only un-processed requests are shown here. The interface is interactive and fast, powered by JavaScript to handle large lists without lag. This is the first stop for an administrator during active relief operations

MEMBER ID	RELIEF TYPE	FLOOD LEVEL	STATUS	ACTION
M02	Food	high	Accepted	View More Details
M02	Water	medium	Accepted	View More Details
M05	Food	high	Accepted	View More Details
M08	Water	high	Accepted	View More Details
M010	Shelter	high	Accepted	View More Details
M012	Water	high	Accepted	View More Details
M014	Food	high	Accepted	View More Details
M018	Medicine	medium	Accepted	View More Details
M020	Water	high	Accepted	View More Details
M010	Water	medium	Accepted	View More Details

Figure 11: Accepted Requests

The Accepted Requests interface is a record of all aid applications that have been approved by the management team. When an administrator "Accepts" a request, the backend updates the database status field to "Accepted". This list provides an actionable plan for logistics teams to begin the distribution of food, water, and medicine. It serves as a reliable log of who is receiving aid, helping to prevent duplicate resource allocation. Each entry includes the contact number and address to facilitate the physical delivery of relief. The interface helps admins monitor ongoing operations and ensure that help is reaching the people in a timely manner. This view is essential for the "Reporting and Monitoring" core functionality of the system. It reflects the system's effectiveness in turning field data into a completed relief action.

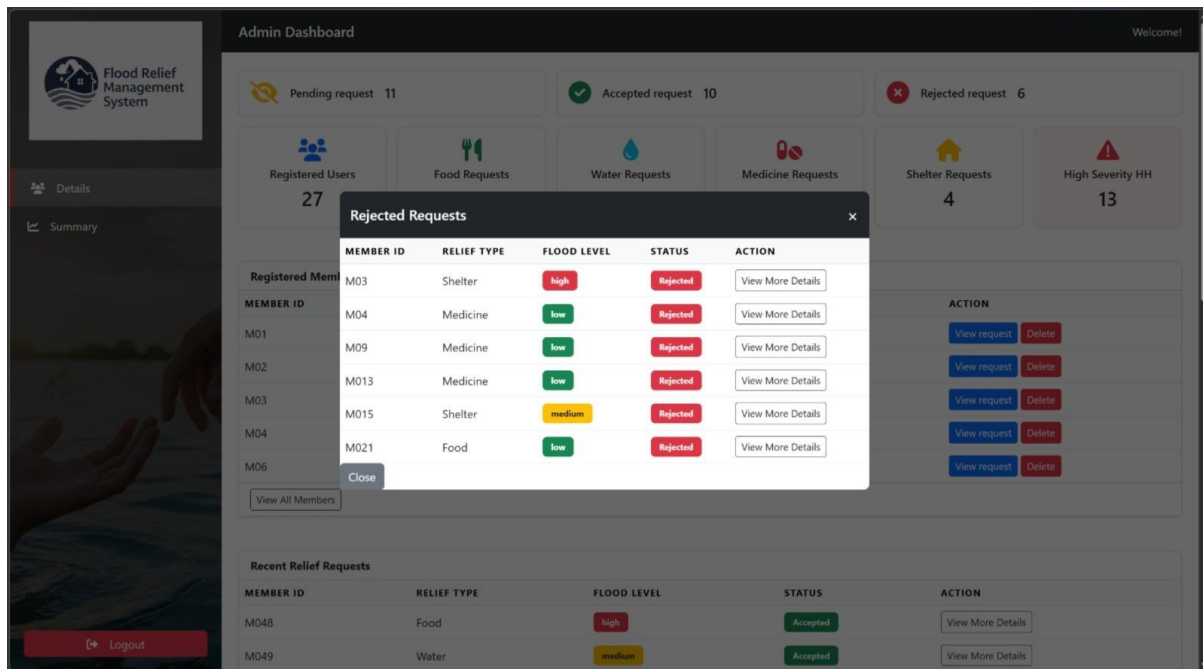


Figure 12: Rejected Requests

This interface lists requests that have been reviewed and determined to be ineligible or unnecessary. A request might be rejected if it is a duplicate or if the information provided is found to be incorrect. The backend marks these records as "Rejected" in the Request table, moving them out of the "Pending" queue. This page is an important part of the system's accountability and data integrity measures. It ensures that the "Accepted" list is not cluttered with invalid requests that would waste resources. Administrators can review these rejections later if more information is provided by the user. The page follows the same structured design as the other admin lists for consistency. It is part of the "Request Management" core functionality of the backend logic. Keeping these records allows for a complete analysis of the disaster response lifecycle.

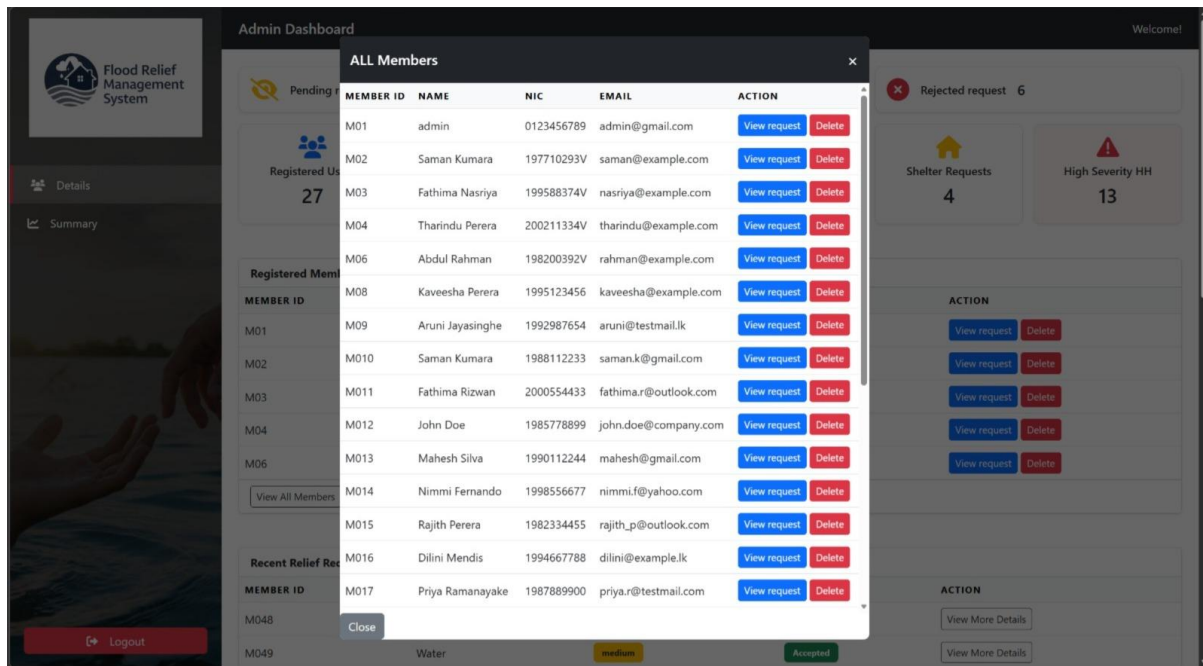


Figure 13: View All Members

The View All Members interface provides administrators with a complete database of every person registered on the platform. It displays key user information such as Name, Email, and NIC, which is pulled directly from the "User" table. Admins can use this page to manage accounts, including updating details or deleting inactive users as part of CRUD operations. This module is critical for identity verification during a disaster to ensure resources are going to real individuals. The interface allows for a broad overview of the community size and the total number of people potentially affected. It is a foundational part of "User Management" within the backend logic. The list is organized and easy to search, ensuring admins can find a specific member's contact info quickly. This page ensures the system's "memory" is accurate and well-maintained.

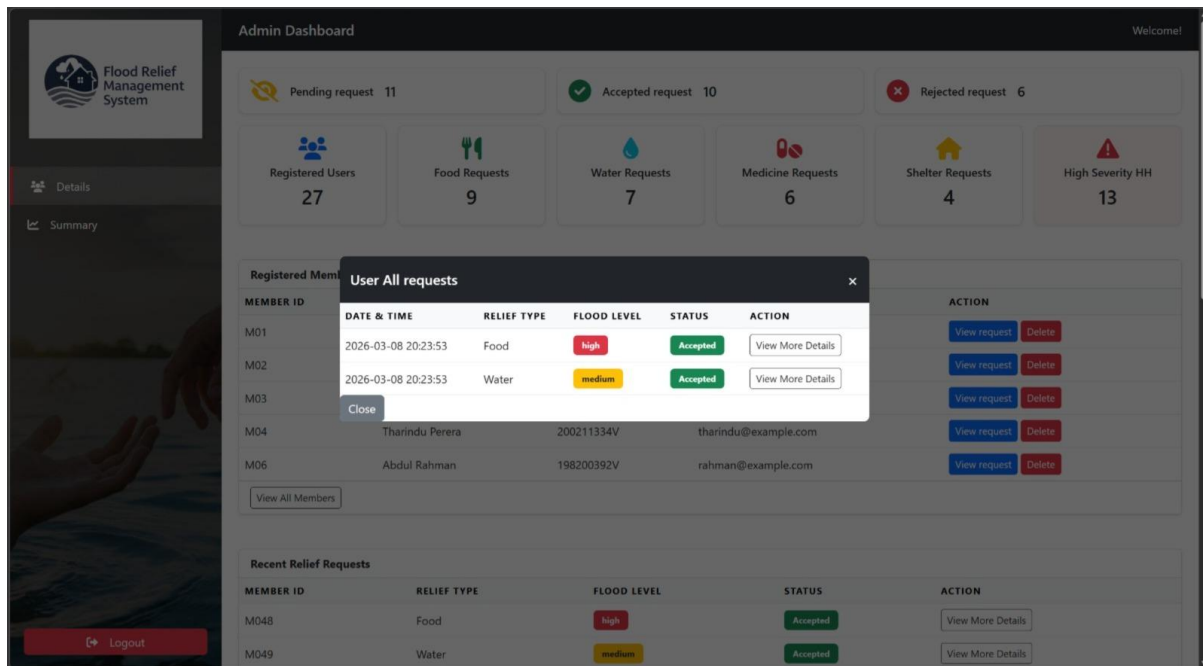


Figure 14:User All Requests

This global view presents every single relief request submitted to the system, regardless of its current status. It combines Pending, Accepted, and Rejected requests into one massive chronological list. Administrators use this interface for long-term historical analysis and to spot overall trends in the disaster. It provides a total "Single Source of Truth" by showing the entire history of the crisis in one place. Each row is linked to a specific user via the user_id foreign key, allowing admins to see the full context of a person's needs. The backend fetches all records from the Request table and presents them in a clean, professional table. This interface is key for generating the "Statistical Reports" that help monitor the entire relief activity. It allows for a global oversight that is impossible with paper-based systems.

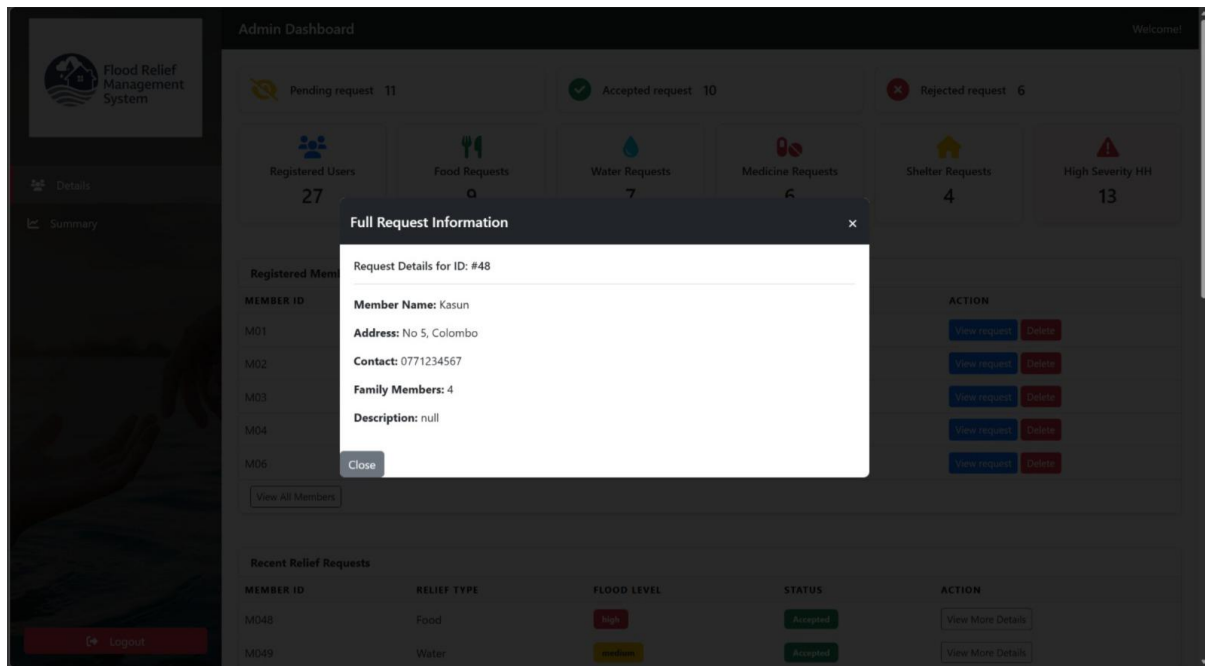


Figure 15: Full Requests Information

This interface is the administrator's version of the detailed request view, optimized for decision-making. It displays all the data from the Request table, including family size, severity, and the user's personal description. This is where the admin decides whether to "Accept" or "Reject" the request. The page includes interactive buttons that trigger the backend logic to update the database status. It emphasizes the "High Severity" label to ensure the admin understands the urgency before taking action. This view prevents errors by showing the requester's contact name and phone number clearly for final verification. It is a prime example of the "Drill-Down" feature, moving from a list to a specific, detailed record. The design is professional and responsive, allowing for quick processing on any device. This is the most critical screen for turning an application for help into a real action.

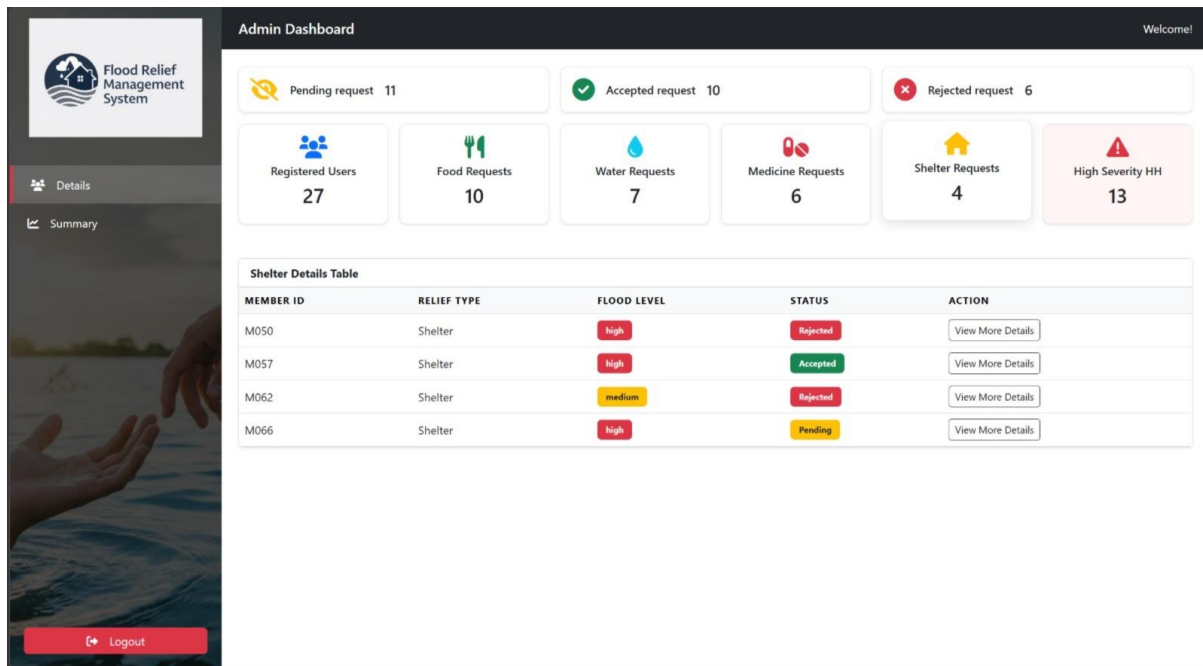


Figure 16: Admin Dashboard

This dashboard interface serves as the analytical hub for coordinating the entire flood response. It features the "Drill-Down" capability, which is the standout feature of the project. The dashboard displays interactive summary counts for different relief types and severity levels. Admins can see the "Big Picture" (total requests) and then click to see the "Small Details" (specific users). This interface is designed to make disaster response faster by providing instant clarity. The backend performs complex grouping and counting operations to provide these numbers in real-time. JavaScript ensures that these analytical views switch instantly without the page ever reloading. It acts as a bridge between the raw field data and the operational command. This module is the primary way the system achieves its goal of "Seeing the Big Picture".

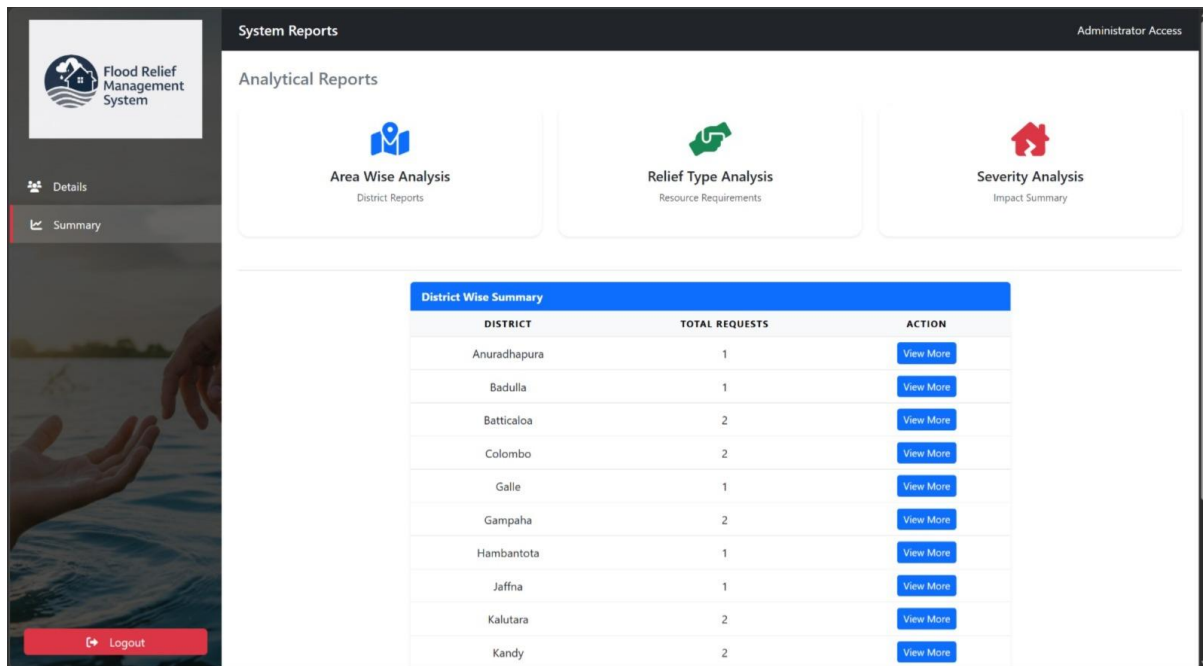
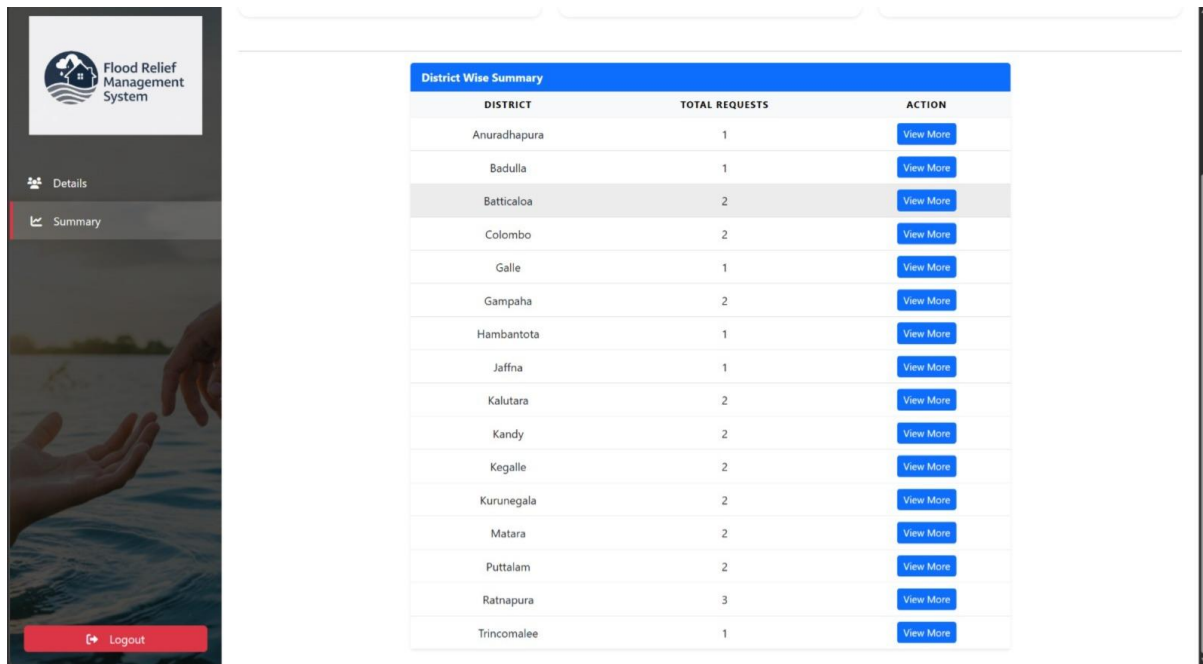


Figure 17: System Summery

The System Summary report provides a high-level statistical overview of the entire project's progress. It displays the total number of processed requests and the current counts of accepted versus rejected applications. This page is intended for senior officials who need to understand the scale of the disaster at a glance. The backend retrieves this data from both the User and Request tables to create a complete picture of the situation. It helps in monitoring the overall performance and effectiveness of the relief operations. The summary is updated dynamically as new data enters the MySQL database. This report is part of the system's "Reporting and Monitoring" core functionality. It provides a sense of clarity during the chaos of a natural disaster. The professional layout is optimized for readability under pressure.



District Wise Summary		
DISTRICT	TOTAL REQUESTS	ACTION
Anuradhapura	1	View More
Badulla	1	View More
Batticaloa	2	View More
Colombo	2	View More
Galle	1	View More
Gampaha	2	View More
Hambantota	1	View More
Jaffna	1	View More
Kalutara	2	View More
Kandy	2	View More
Kegalle	2	View More
Kurunegala	2	View More
Matara	2	View More
Puttalam	2	View More
Ratnapura	3	View More
Trincomalee	1	View More

Figure 18: All district Summary Report

This report categorizes relief requests by the district they originated from, providing a geographic overview of the crisis. It allows administrators to identify which regions of the country are the most heavily affected. The backend groups the database records by the district field to generate these summary counts. This analysis is crucial for directing large-scale resources like food trucks or rescue boat fleets to the right areas. It helps prevent the misallocation of aid to less-affected districts while others are in crisis. The report is presented in a clear table format that can be sorted for quick analysis. This is a key output of the "Area-Wise Analysis" module. It helps decision-makers manage the supply chain by seeing exactly where the demand is. This report ensures that the relief effort is geographically balanced and fair.

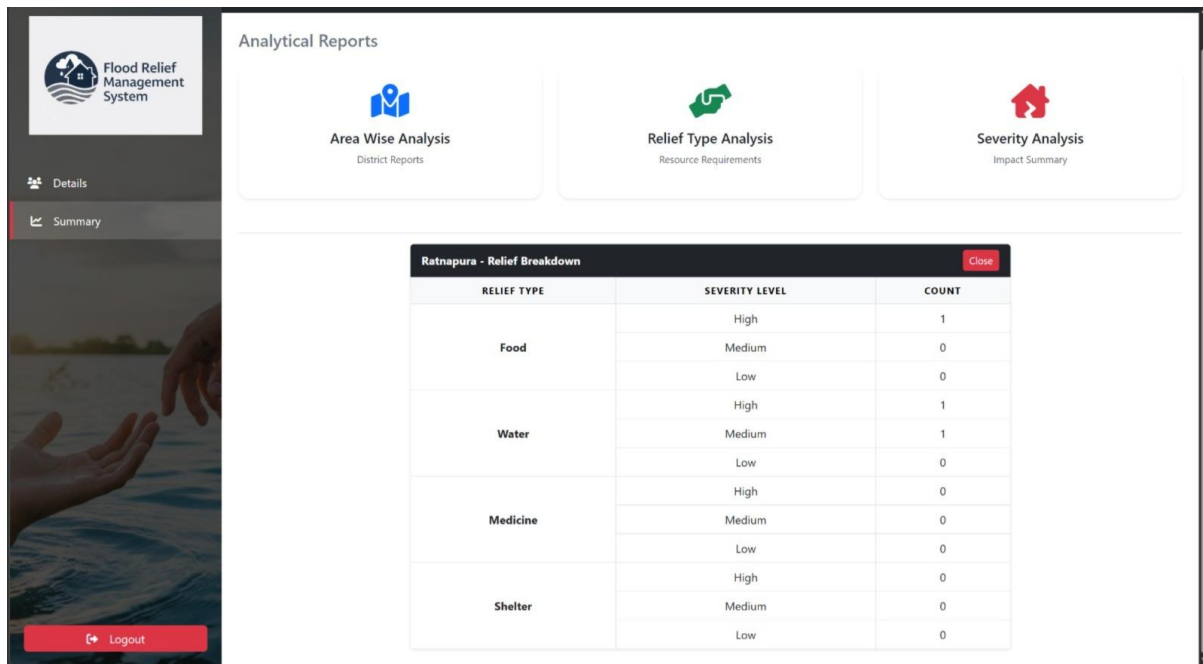


Figure 19:Area-Wise Summary Report

The Area-Wise Summary Report drills down deeper than the district level, focusing on local areas like Divisional Secretariats or GN Divisions. This granular view helps ground teams identify specific "hotspots" of flood activity within a district. The backend uses the precise location attributes from the Request table to generate this report. It is essential for organizing localized response efforts, such as establishing a neighborhood medical camp. This report helps in prioritizing the "Small Details" that make a difference in local communities. The interface is responsive, allowing field officers to use this localized data while on the move. It is part of the system's strategy to provide both high-level and deep-dive analysis. This ensures that even small, isolated groups of people are not forgotten during the relief effort.

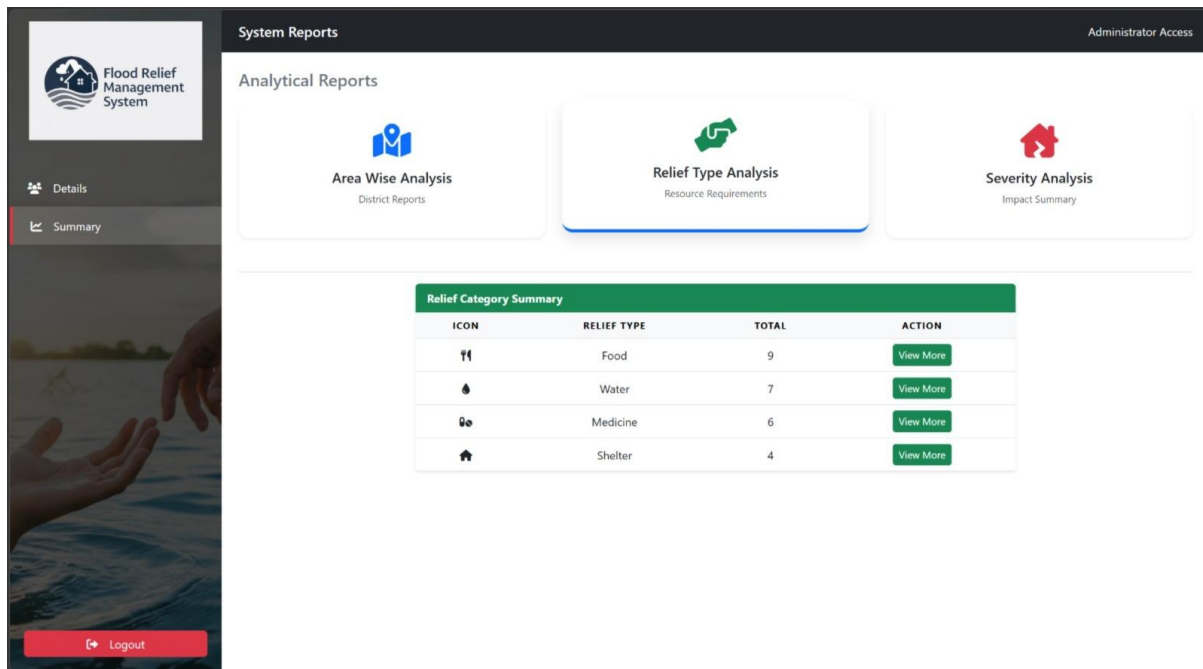


Figure 20: Relief-Wise Summary Report

This report analyzes the types of aid requested across the entire system (Food, Water, Medicine and Shelter). It gives administrators a clear count of each resource type, helping them manage inventory and supply chains. The backend filters the MySQL database by the relief type attribute to create this summary. For example, a sudden spike in "Medicine" requests in a specific area could indicate an emerging health crisis. This information is vital for communicating with NGOs and donors about what supplies are most needed. The interface provides a statistical breakdown that turns thousands of requests into a simple resource plan. It is a core part of the "Relief Type Analysis" output. This report ensures that the system provides the right help to the right people at the right time.

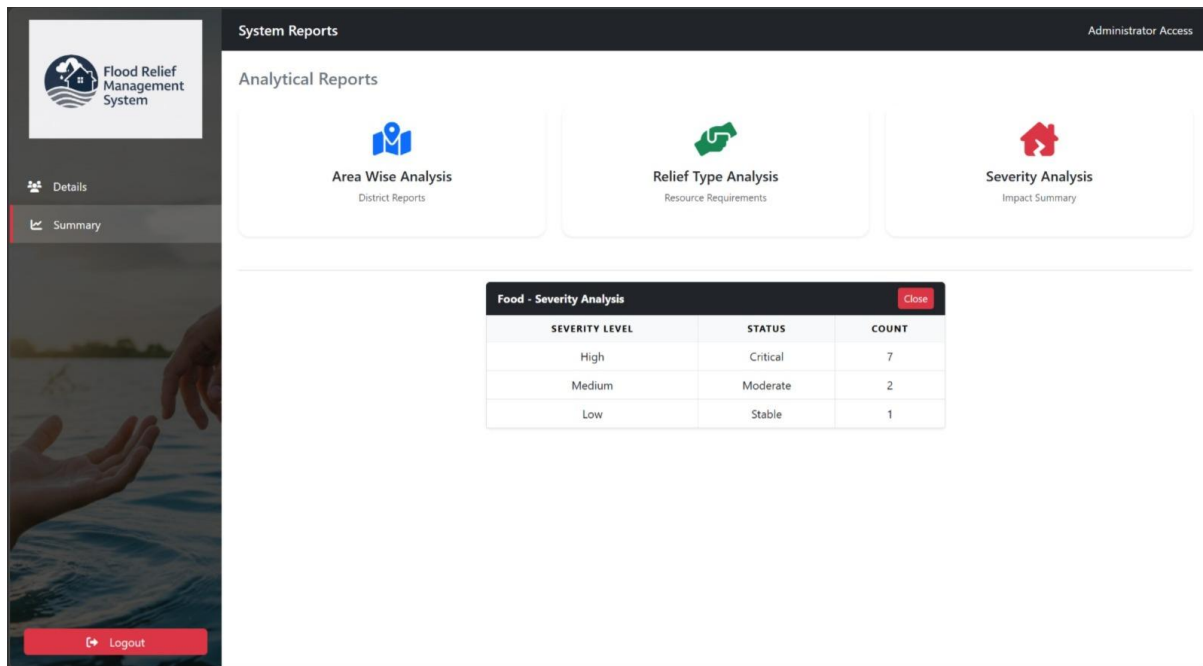


Figure 21: Food-Severity Analysis

The Food-Severity Analysis report ranks all incoming requests based on the reported urgency:

High, Medium or Low.

This is the most critical analytical tool for saving lives, as it allows for immediate prioritization. The backend pulls data from the severity levels column in the Request table to generate this ranking. It ensures that "High Severity" cases—where people might be in immediate danger—are visible at the top of the list. Admins use this report to deploy life-saving resources like boats and medical teams to the most dangerous zones first. The interface provides a clear summary count for each severity level to help gauge the overall danger. This analysis is fundamental to the system's goal of "Prioritizing Those in Danger". It transforms a chaotic list of requests into a focused, prioritized plan of action.

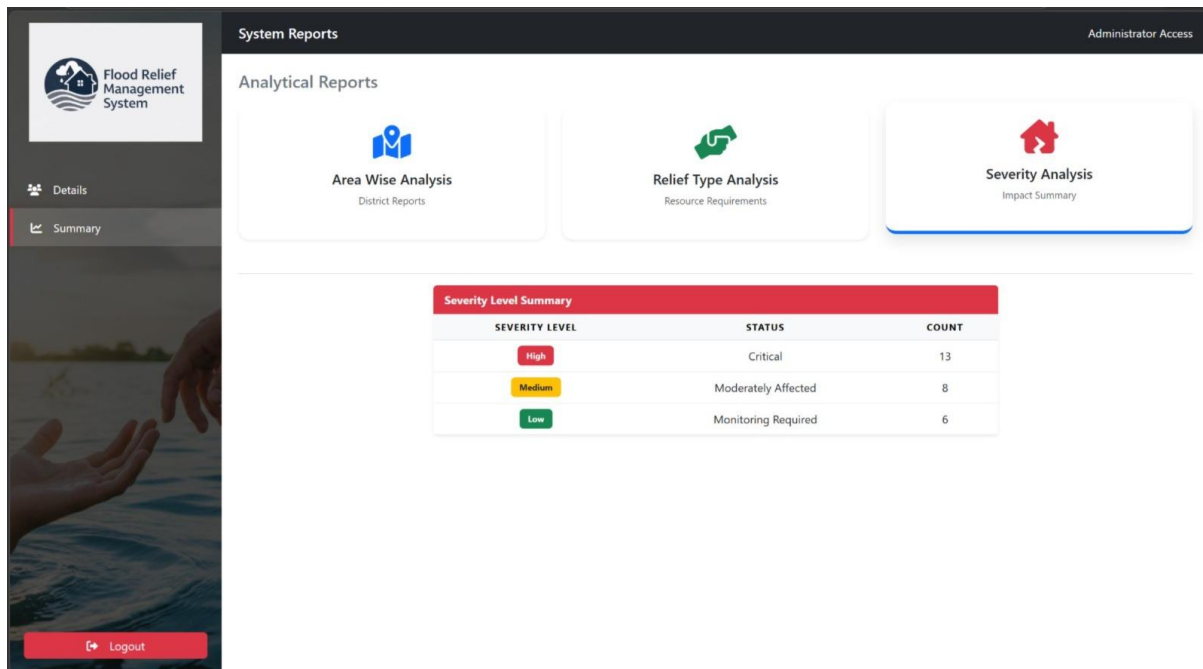


Figure 22:Severity Level Summary

This summary page provides a statistical breakdown of the total count of High, Medium, and Low severity cases. It offers a high-level view of the overall intensity of the flood disaster across all regions. The backend calculates these totals in real-time, allowing for an up-to-the-minute assessment of the situation. This summary is essential for strategic planning, helping officials decide if more national resources are required. It is part of the system's mission to organize "messy data into clear, organized reports". The layout is simple and uses Bootstrap to ensure the numbers are easily readable on any screen. This report acts as a simplified dashboard for senior decision-makers to understand the scope of the emergency. It ensures that the disaster management team is always aware of the most critical needs.

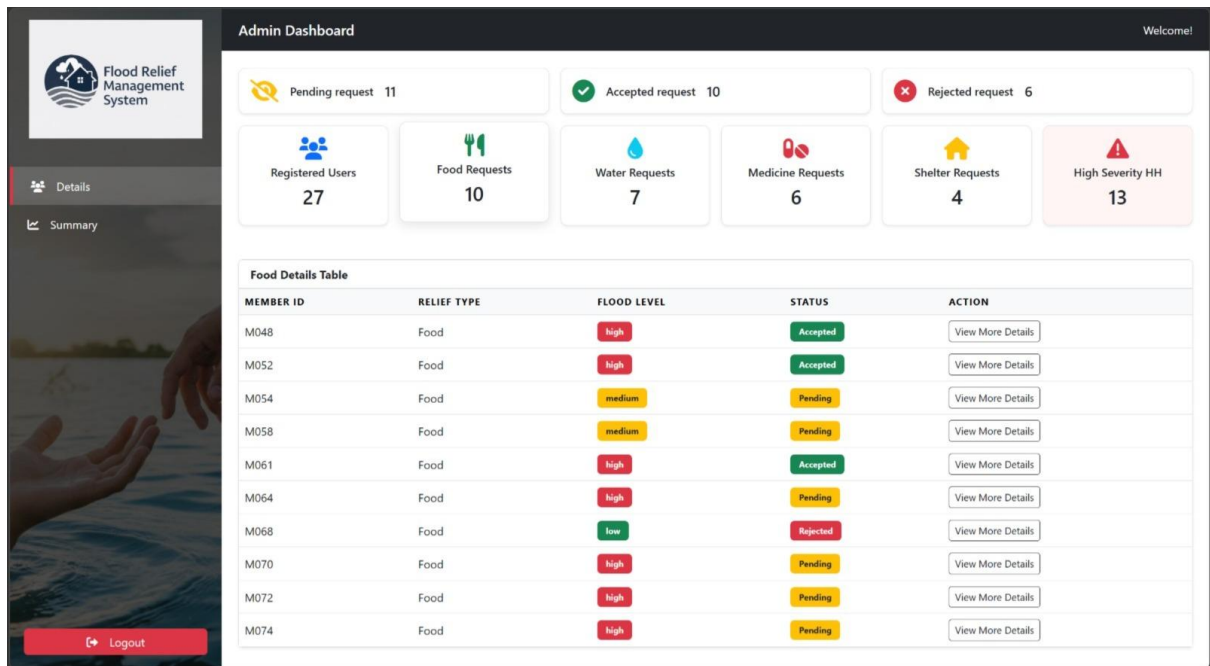


Figure 23: Food Details Table

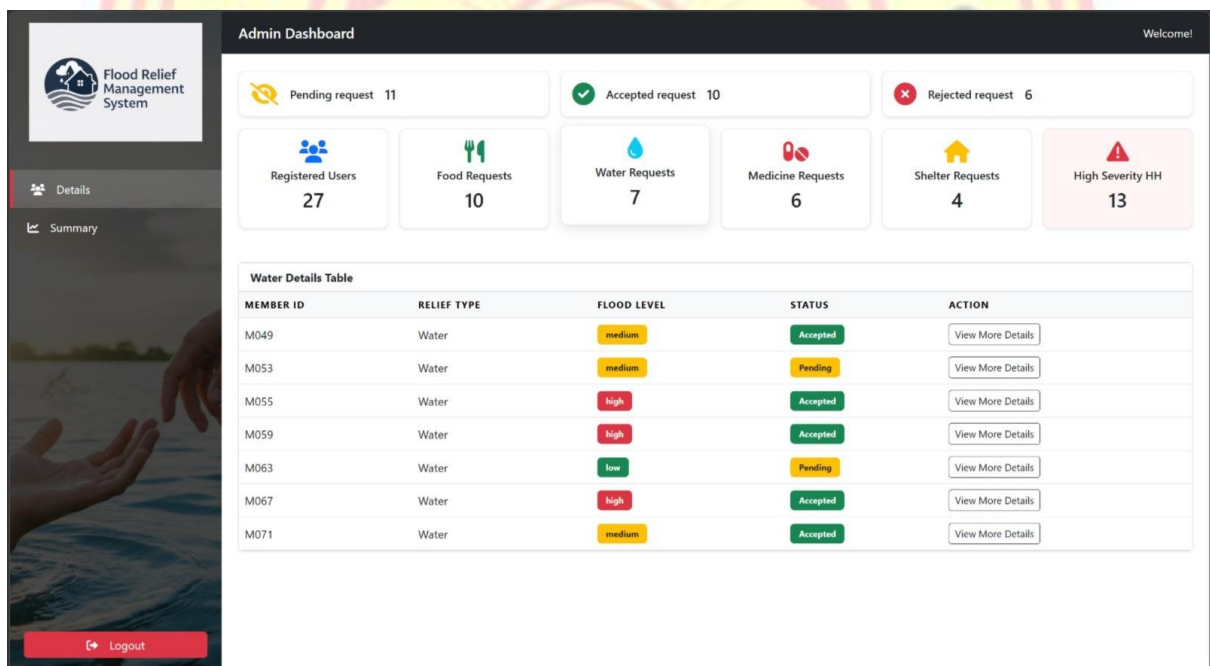


Figure 24: Water Details Table

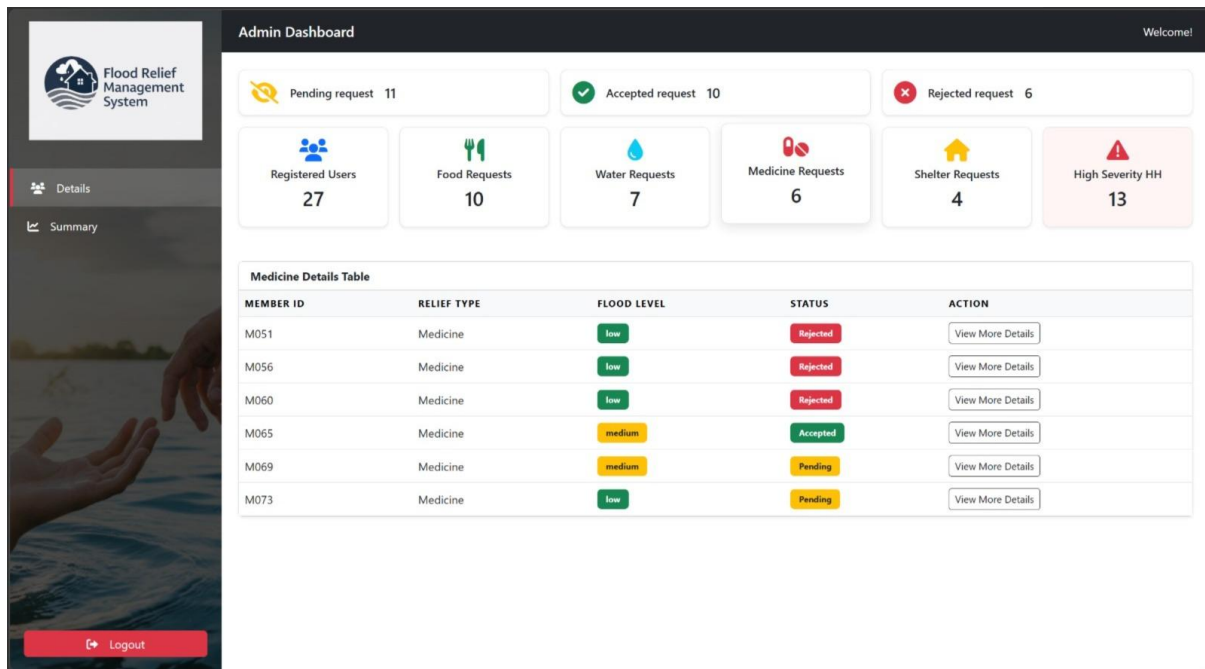


Figure 25:Medicine Details Table

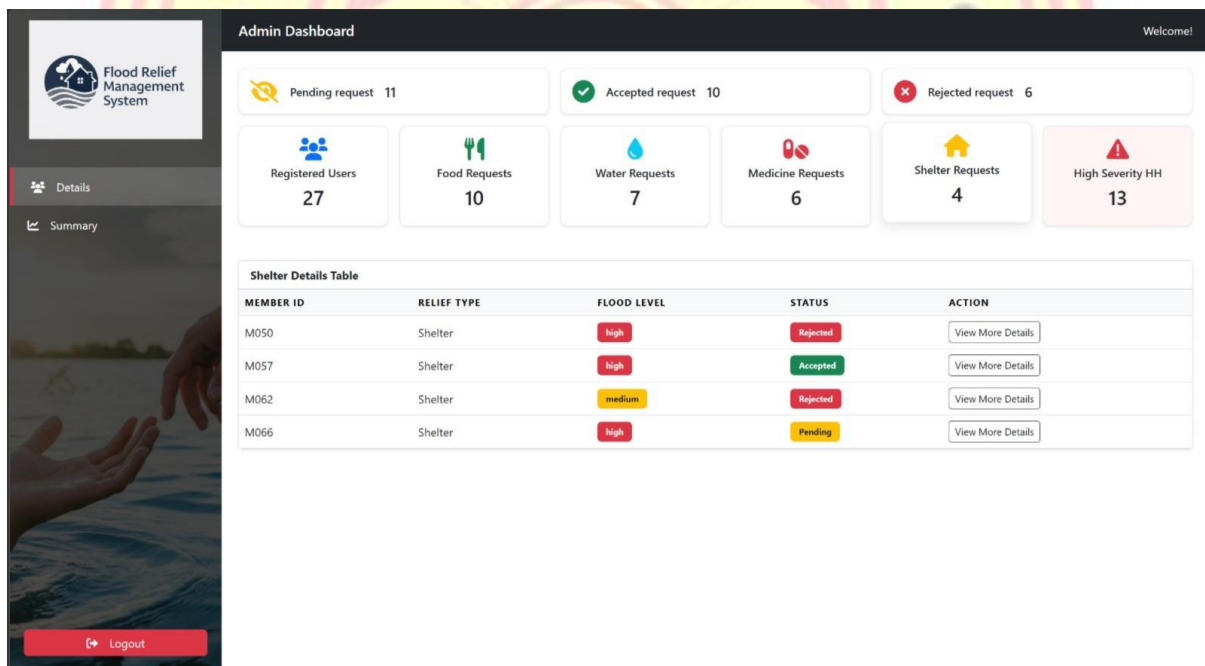
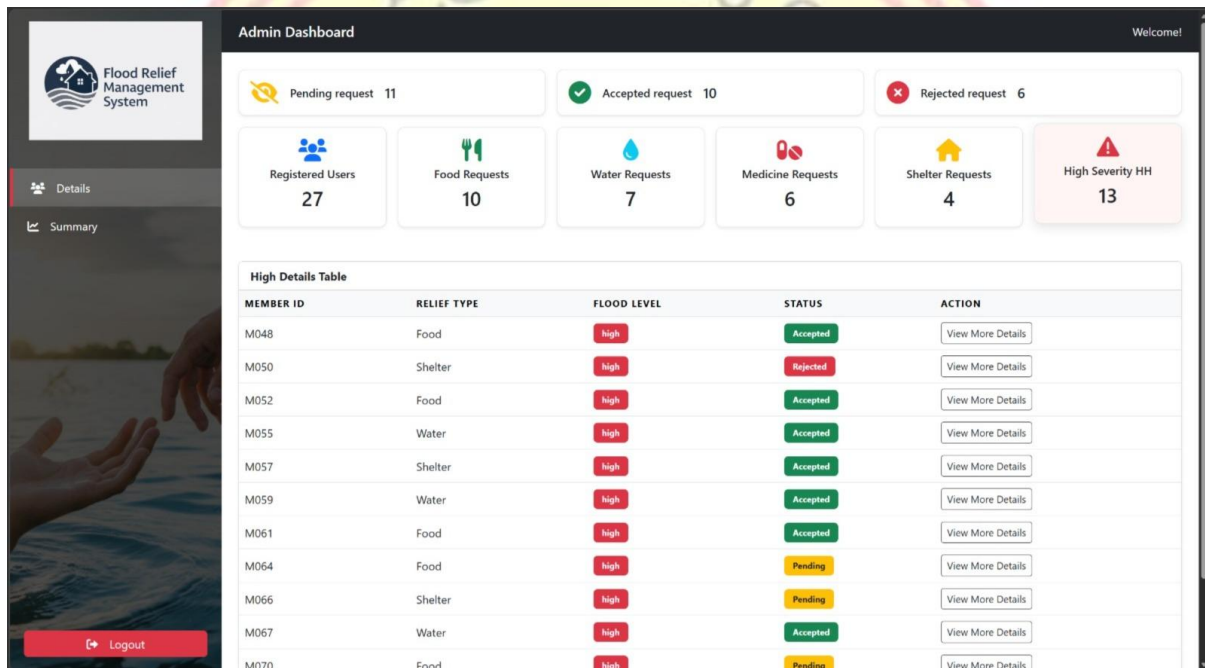


Figure 26:Shelter Details Table

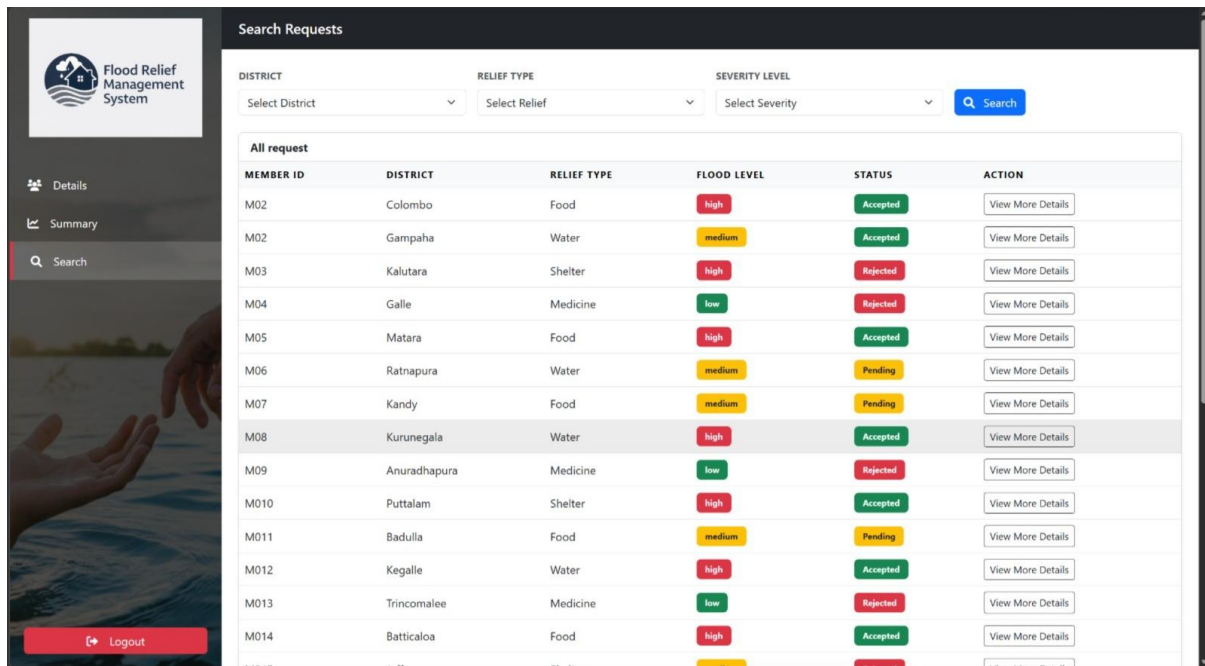
These specialized tables isolate requests for one specific type of relief, such as Food (Fig 23), Water (Fig 24), Medicine (Fig 25), or Shelter (Fig 26). These isolated lists allow dedicated distribution teams to focus solely on their area of expertise. For example, the medical team can use Figure 25 to find everyone needing healthcare without seeing food or water requests. The backend filters the database by the `relief_type` foreign key to generate these specific views. Each table includes the name, location, and contact information for the requester to facilitate delivery. This organization is part of the "Drill-Down" strategy, providing the "small details" for specific operations. It greatly improves the efficiency of field teams by removing irrelevant data. These reports ensure that resources are handled by the right specialized team for each category



MEMBER ID	RELIEF TYPE	FLOOD LEVEL	STATUS	ACTION
M048	Food	high	Accepted	View More Details
M050	Shelter	high	Rejected	View More Details
M052	Food	high	Accepted	View More Details
M055	Water	high	Accepted	View More Details
M057	Shelter	high	Accepted	View More Details
M059	Water	high	Accepted	View More Details
M061	Food	high	Accepted	View More Details
M064	Food	high	Pending	View More Details
M066	Shelter	high	Pending	View More Details
M067	Water	high	Accepted	View More Details
M070	Food	high	Pending	View More Details

Figure 27:List of High Severity Requests

This lists only the "High Severity" requests that require immediate action. It serves as a direct rescue list for emergency responders, highlighting people in the most dangerous zones. The backend strictly filters the Request table to show only the most critical cases, ensuring they are never buried in the data. Each entry provides the precise location and contact details needed for a swift rescue mission. This list is the ultimate realization of the project's goal to prioritize life-saving resources. It is designed to be accessible on mobile devices for rescue teams standing out in the rain. By isolating these cases, the system ensures that those in the greatest danger are the first to be helped. This interface provides the most urgent "Plan of Action" during the peak of a disaster.



Search Requests

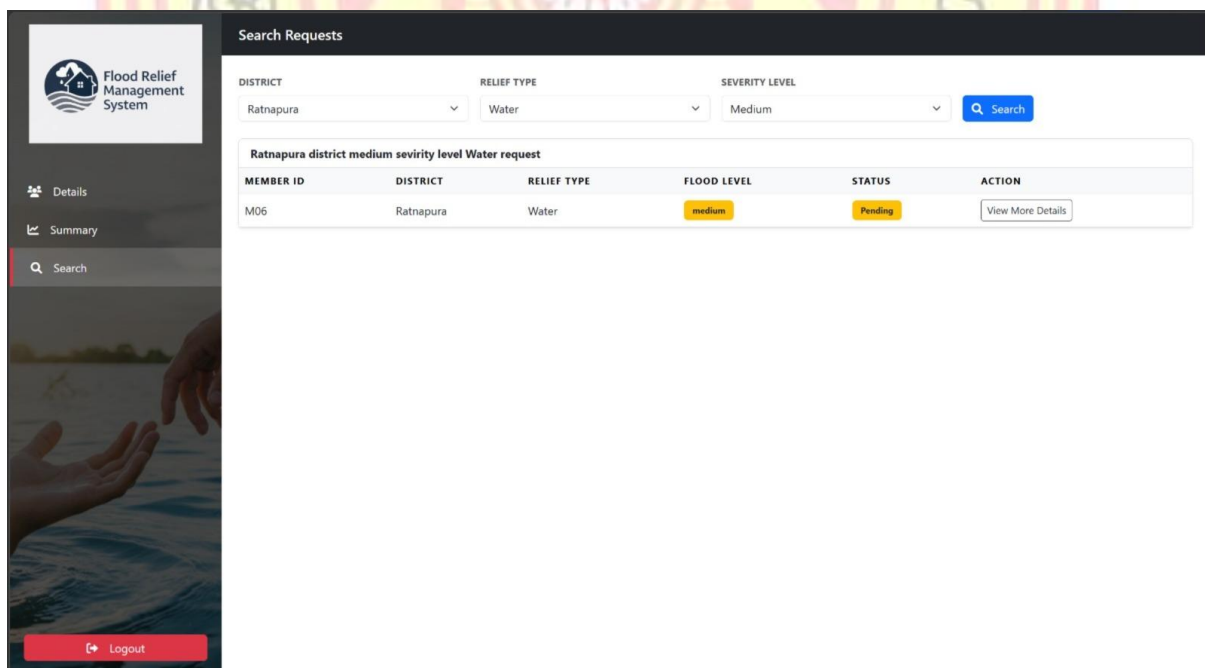
DISTRICT: Select District | RELIEF TYPE: Select Relief | SEVERITY LEVEL: Select Severity | Search

All request

MEMBER ID	DISTRICT	RELIEF TYPE	FLOOD LEVEL	STATUS	ACTION
M02	Colombo	Food	high	Accepted	View More Details
M02	Gampaha	Water	medium	Accepted	View More Details
M03	Kalutara	Shelter	high	Rejected	View More Details
M04	Galle	Medicine	low	Rejected	View More Details
M05	Matara	Food	high	Accepted	View More Details
M06	Ratnapura	Water	medium	Pending	View More Details
M07	Kandy	Food	medium	Pending	View More Details
M08	Kurunegala	Water	high	Accepted	View More Details
M09	Anuradhapura	Medicine	low	Rejected	View More Details
M10	Puttalam	Shelter	high	Accepted	View More Details
M11	Badulla	Food	medium	Pending	View More Details
M12	Kegalle	Water	high	Accepted	View More Details
M13	Trincomalee	Medicine	low	Rejected	View More Details
M14	Batticaloa	Food	high	Accepted	View More Details

Figure 28: Search Requests Dashboard

Here prints all the requests. By organizing requests by severity and relief type, authorities can prioritize high-risk areas and ensure that resources are allocated where they are most urgently needed.



Search Requests

DISTRICT: Ratnapura | RELIEF TYPE: Water | SEVERITY LEVEL: Medium | Search

Ratnapura district medium severity level Water request

MEMBER ID	DISTRICT	RELIEF TYPE	FLOOD LEVEL	STATUS	ACTION
M06	Ratnapura	Water	medium	Pending	View More Details

Figure 29: View Searched Requests

After filtering requests, we can have an interface like this. This demonstrates the system's search result state. While the previous screen showed the entire database, this interface displays the system's ability to isolate specific data points based on user-defined criteria.

3.4. Summary of the Project

The Flood Relief Management and Analytical Reporting System is a digital platform designed to streamline disaster response by transforming large volumes of chaotic field data into clear, actionable reports. Its primary goal is to address "information overload" during floods, ensuring that rescue teams can identify and prioritize those in the most urgent need.

Key Features:

- **"Drill-Down" Dashboard:** A standout feature that presents high-level summaries (e.g., total requests in a district) and allows administrators to click for deep-dive specifics (e.g., exact medical supply needs) without reloading the page.
- **Categorized Reporting:** Automatically organizes relief needs into categories like Food, Water, Medicine, and Shelter, ranked by severity from High to Low.
- **Request Management:** Enables users to create, view, and update their relief requests while allowing admins to accept or reject them.

Technical Implementation:

The system is built on a modern "Client-Server" stack:

- **Backend (PHP & MySQL):** PHP handles the "heavy lifting" logic and calculations, while MySQL acts as the system's "memory" for storing all requests and user data.
- **Frontend (HTML, CSS, & Bootstrap):** These technologies create a professional, device-agnostic interface that functions across various screen sizes.
- **Interactivity (JavaScript):** Acts as the "brain" of the interface, allowing for instant view-switching and snappy interactivity.

Target Users:

1. **Disaster Management Admins:** Who use the analytical reports to manage overall relief strategy and coordinate large-scale aid distribution.
2. **Field Users:** On-the-ground teams who can check requirements and update request statuses directly from their mobile devices while in disaster zones.

4.Each Group Member's Contributions

Index Number	Relevant Contribution	Signature
AS20240555	Full frontend of Admin dashboard and overview backend and search of admin dashboard , Merged branches, finalized and fixed bugs	
AS20240414	Frontend and Backend of Admin dashboard	
AS20240617	Frontend and Backend of Login and Registration, Documentation about the system	
AS20240634	Frontend of Main page and Backend of Summary in Admin dashboard	

Github Repository link:

<https://github.com/SewmithUSJ/Flood-Relief-Management-System.git>